Microprocessor Design Final Project Report

Final Project

顾洋铭12312801(32%)

周晓俊12311815(26%)

邱泉友 12311801(22%)

叶峰齐12311802(20%)

1. **Implementation of a five-stage pipeline processor with Hazard Handling**
   - 1.1 Introduction
   - 1.2 Hardware architecture
   - 1.3 Hazard handling techniques
   - 1.4 Implementation details
   - 1.5 Test code and simulation waveforms
   - 1.6 Onboard test result
2. **Design and Benchmarking of a RISC-V Processor**
   - 2.1 Introduction
     - 2.1.1 Concise Overview of RISC-V ISA (RV32I Focus)
     - 2.1.2 Key Differences Between RISC-V (RV32I) and ARM
   - 2.2 Description of CPU Modules and Implementation
     - 2.2.1 Single-Cycle RISC-V (RV32I) CPU Implementation
       - 2.2.1.1 Hardware architecture

2.2.1.2 Top.cpu module

2.2.1.3 Control Unit module

2.2.1.4 PC module

2.2.1.5 InsMEM module

2.2.1.6 RegFile module

2.2.1.7 ALU module

2.2.1.8 Extend module

2.2.1.9 data memory module

2.2.2 CPU simulation waveforms of single cycle RISC-V (RV32I) CPU

2.2.3 Multi-Cycle RISC-V (RV32I) CPU Implementation(pipline)

2.2.4 CPU simulation waveforms of multi cycle RISC-V (RV32I) CPU

2.2.5 Implementation challenges and reflections

3. Bonus

---

1. Implementation of a five-stage pipeline processor with Hazard Handling

1.1 Introduction

The five-stage pipelined architecture represents a fundamental design paradigm in contemporary microprocessor design. This sophisticated approach partitions the sequential execution of individual instructions into five discrete and largely autonomous processing phases, thereby establishing instruction-level concurrency that dramatically enhances computational throughput. The architectural framework comprises the following meticulously orchestrated stages:

Instruction Fetch (IF): During this initial phase, the processor retrieves the subsequent instruction from the instruction memory subsystem, simultaneously incrementing the program counter to maintain execution flow. This stage may incorporate sophisticated branch prediction mechanisms and instruction cache hierarchies to optimize fetch efficiency.

Instruction Decode (ID): The retrieved instruction undergoes comprehensive decoding in this critical phase. The control unit meticulously interprets the instruction's opcode and

functional fields, generates appropriate control signals for subsequent pipeline stages, reads operand values from the register file, and performs necessary immediate value expansion and sign extension operations.

Execute (EX): This computational core phase performs the instruction's essential arithmetic, logical, or address calculation operations through dedicated execution units. For conditional branch instructions, this stage evaluates the branch condition and computes potential target addresses, while for memory operations it calculates effective memory addresses.

Memory Access (MEM): Instructions requiring data memory interactions, such as load and store operations, perform their actual memory transactions during this dedicated phase. The memory subsystem handles read requests for load operations and write operations for store instructions, with potential cache hierarchy involvement for performance optimization.

Write Back (WB): In this concluding phase, computational results or retrieved data from memory operations are committed to the architectural register file, making these values available for consumption by subsequent instructions in the execution stream.

Through this carefully engineered temporal partitioning, the pipeline enables concurrent processing of multiple instructions at different completion stages during each clock cycle, effectively transforming a predominantly sequential execution model into a highly parallelized throughput-oriented architecture. This parallelization yields substantial improvements in instructions-per-cycle (IPC) metrics and overall system performance.

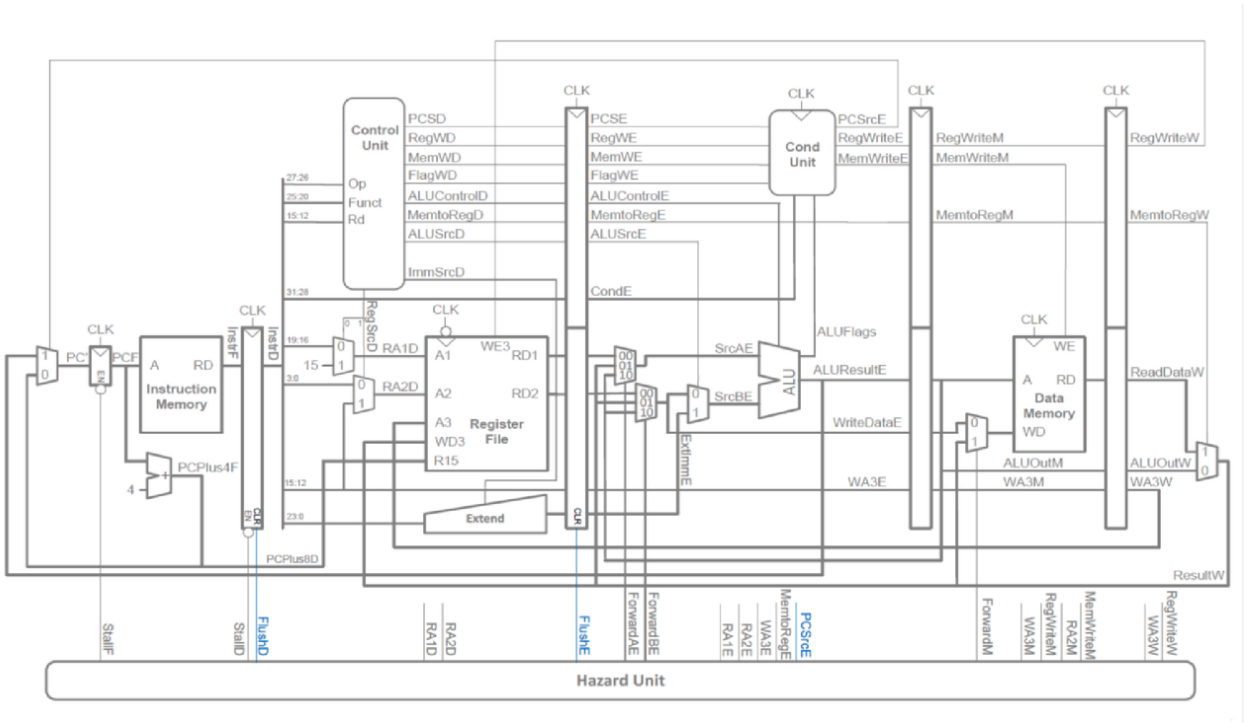
However, this architectural sophistication introduces several inherent challenges that demand sophisticated resolution mechanisms. Data hazards emerge when instructions exhibit read-after-write, write-after-read, or write-after-write dependencies, potentially compromising program correctness. Control hazards arise from branch instructions disrupting the sequential instruction flow, creating pipeline bubbles. Structural hazards occur when multiple instructions simultaneously contend for limited hardware resources.

Modern processors employ an arsenal of advanced techniques to mitigate these challenges, including: sophisticated **data forwarding/bypassing networks that directly route results between pipeline stages, speculative execution with branch prediction algorithms, out-of-order execution capabilities, register renaming to eliminate false dependencies, branch target buffers for rapid address calculation, and precise interrupt handling mechanisms. These optimization strategies collectively enable the five-stage pipeline to deliver the performance benefits of parallelism while maintaining architectural correctness and program consistency.

1.2 Hardware architecture

In this section, I will describe the modules included at each level, as well as some of the changes compared to the Single cycle. The overall block diagram of the five-stage pipeline is

shown in the figure below:



The entire architecture is divided into five parts by four vertical lines, which is referred to as the five-level architecture. Instructions are sent in one after another for processing, just like a production line, so it is collectively called the five-level pipeline ARM. IF is responsible for fetching instructions, so the program counter is placed at this level. It fetches instructions by continuously incrementing PC by 4. The ID is responsible for decoding. The Decoder module is placed at this level to implement the decoding function, and then the decoded instructions are read through the RF module. While ALU and Condlogic are assigned to the EX stage, the ALU module provides the corresponding computing capabilities for the EX stage. Then, the Condlogic module can promptly update the condition codes and determine whether the condition should be executed within the same stage. The Datamemory is assigned to the MEM stage and is responsible for handling the LDR and STR instructions. The final remaining component is WB, which is responsible for writing the calculation results back to the RF module. The 5-stage pipeline ARM differs from the single-cycle architecture in many minor aspects.

Among them, the most important point is that for R15, it is no longer PC + 8, but PC + 4. We have removed the second PCPLUS4F adder. Another point is that Instr[15:12] is no longer directly written to A3. Instead, it needs to go through the EX, MEM, and WB stages in sequence before being written, with the aim of achieving consistency in timing and preventing disorder.

1.3 Hazard handling techniques

Data hazards represent a fundamental challenge in pipelined processors, arising when instructions at different stages of execution exhibit interdependencies that threaten program correctness. These hazards manifest as situations where the flow of data between instructions is disrupted by the processor's parallel execution model. The most prevalent form is the Read-

After-Write hazard, where a consuming instruction attempts to access a register value before the producing instruction has completed its write operation. This creates a critical path dependency that must be carefully managed to ensure computational accuracy.

To address data hazards, modern processors employ sophisticated forwarding mechanisms, also known as bypass networks. These hardware pathways allow results to be directly transmitted from the execution units of producer instructions to the input stages of consumer instructions, effectively "short-circuiting" the normal pipeline flow. For instance, when an arithmetic operation completes in the Execute stage, its result can be immediately forwarded to subsequent instructions still in their Decode stage, eliminating the need to wait for formal register file updates. More advanced processors implement multi-level forwarding networks capable of handling complex dependency chains across multiple pipeline stages. When forwarding proves insufficient—particularly for load operations immediately followed by dependent instructions—processors may strategically insert pipeline stalls or bubbles, temporarily halting progression to maintain correctness while minimizing performance degradation.

Control hazards emerge from the disruptive nature of branch instructions on the sequential flow of program execution. These hazards occur because processors must make fetch decisions before knowing whether a branch will be taken or what its target address might be. When a branch instruction enters the pipeline, subsequent instructions that have already been fetched may need to be discarded if the branch direction differs from initial predictions. This creates "pipeline bubbles"—cycles where no useful work occurs—significantly impacting processor throughput and efficiency.

Contemporary processors combat control hazards through predictive execution strategies. Branch prediction algorithms, ranging from simple static predictors to complex dynamic neural predictors, attempt to forecast branch outcomes before they're definitively known. Complementary to prediction, branch target buffers store previously calculated branch addresses for rapid retrieval. More aggressive approaches include speculative execution, where processors proceed with predicted instruction paths while maintaining the ability to roll back if predictions prove incorrect. Some architectures employ delayed branch techniques, where instructions following branches are always executed regardless of branch outcome, though this requires careful compiler orchestration. For unconditional branches, early resolution mechanisms in the Decode stage can minimize disruption, while sophisticated return address stacks manage function call returns with high accuracy.

Structural hazards originate from resource contention within the processor architecture, occurring when multiple instructions simultaneously require access to the same physical hardware component. These hazards reflect fundamental limitations in processor design where shared resources — such as functional units, memory ports, or register file access points— cannot serve all requesting instructions concurrently. Common manifestations include multiple

instructions competing for a single arithmetic logic unit, simultaneous memory access requests, or contention for specialized execution units like multipliers or dividers.

Resolution of structural hazards employs both hardware and organizational strategies. Resource duplication represents a direct approach, where critical components like ALUs or memory ports are replicated to reduce contention. Time-multiplexing strategies allocate shared resources across different pipeline stages or clock cycles. Modern superscalar processors implement reservation stations and instruction queues that buffer operations until required resources become available. Compiler techniques like instruction scheduling rearrange code sequences to minimize simultaneous resource demands. More fundamentally, processor architects carefully balance pipeline depth against resource availability, ensuring that the parallelism enabled by pipelining isn't undermined by structural bottlenecks. Some advanced designs incorporate dynamic resource allocation mechanisms that can reconfigure execution units based on workload characteristics, while others employ out-of-order execution to work around temporary resource constraints by progressing independent instructions.

The above is a relatively complete summary. In the code, we mainly use lookahead, STALL, different clock edges, and inserting NOPS to address data hazards. Through early BTA and FLUSH, we solve control hazards. However, structural hazards are not within the scope of this PROJECT, so they will not be handled for now.

1.4 Implementation details

First, present the code for the five-level division. The main task is to divide the signals within the ARM module into five levels and give each level a separate name. The relevant code is as follows:

代码块

```
// Fetch Block:
wire StallF;
wire [31:0] PCF, PC_Plus_4F;
wire [31:0] InstrF;

// Decode Block:
wire StallD, FlushD;
reg [31:0] InstrD;
wire PCSD;
wire RegWD;
wire MemWD;
wire [1:0] FlagWD;
wire [1:0] ALUControlD;
wire MemtoRegD;
wire ALUSrcD;
wire [1:0] ImmSrcD;
```

```
17  wire [2:0] RegSrcD;
18  wire NoWriteD;
19  wire M_StartD;
20  wire MCycleOpD;
21  wire M_WD;
22  wire [3:0] CondD;
23  wire [6:0] shControlD;
24
25  wire [3:0] RA1D, RA2D, WA3D;
26  wire [31:0] RD1D, RD2D;
27  wire [31:0] PC_Plus_8D;
28  wire [23:0] InstrImmD;
29  wire [31:0] ExtImmD;
30
31  // Execute Block:
32  wire StallE, FlushE;
33  wire [1:0] ForwardAE, ForwardBE;
34  reg [31:0] InstrE;
35  reg PCSE;
36  reg RegWE;
37  reg MemWE;
38  reg [1:0] FlagWE;
39  reg [1:0] ALUControlE;
40  reg MemtoRegE;
41  reg ALUSrcE;
42  reg NoWriteE;
43  reg M_StartE;
44  reg MCycleOpE;
45  reg M_WE;
46  reg [3:0] CondE;
47  reg [6:0] shControlE;
48  reg [3:0] RA1E, RA2E, WA3E;
49  reg [31:0] RD1E, RD2E;
50  reg [31:0] ExtImmE;
51  wire PCSrcE;
52  wire RegWriteE;
53  wire MemWriteE;
54  wire M_WriteE;
55  wire [1:0] ShE;
56  wire [4:0] Shamt5E;
57  wire [31:0] ShInE;
58  wire [31:0] ShOutE;
59  wire [31:0] SrcAE, SrcBE;
60  wire [31:0] ALUResultE;
61  wire [3:0] ALUFlagsE;
62  wire [31:0] MCycleResultE;
63  wire M_BusyE;
```

```
wire [31:0] OpResultE;
wire [31:0] WriteDataE;

// Memory Block:
wire FlushM;
wire ForwardM;
reg [31:0] InstrM;
reg RegWriteM;
reg MemWriteM;
reg MemtoRegM;
reg [31:0] OpResultM;
reg [31:0] WriteDataM;
reg [3:0] RA2M, WA3M;
wire [31:0] ReadDataM;

// Write Back Block:
reg [31:0] InstrW;
reg RegWriteW;
reg MemtoRegW;
reg [31:0] ReadDataW;
reg [31:0] OpResultW;
reg [3:0] WA3W;
wire [31:0] ResultW;
```

Then, following the assembly line process, corresponding instances were created for each of the ARM sub-modules, while the FLUSH, STALL instruction was also added (Their logic is very simple. For FLUSH, the corresponding signal is set to 0, while for STALL, the corresponding signal is set to itself.).

代码块

```
Decoder Decoder1(
.Instr(InstrD),
.PCS(PCSD),
.RegW(RegWD),
.MemW(MemWD),
.MemtoReg(MemtoRegD),
.ALUSrc(ALUSrcD),
.ImmSrc(ImmSrcD),
.RegSrc(RegSrcD),
.ALUControl(ALUControlD),
.FlagW(FlagWD),
.NoWrite(NoWriteD),
.M_Start(M_StartD),
.MCycleOp(MCycleOpD),
.M_W(M_WD)
```

```
16  );
17
18  //MC04. Add Datapath
19  assign RA1D = RegSrcD[2]? InstrD[11:8]: (RegSrcD[0]? 4'd15: InstrD[19:16]);
20  assign RA2D = RegSrcD[1]? InstrD[15:12]: InstrD[3:0];
21  assign WA3D = RegSrcD[2]? InstrD[19:16]: InstrD[15:12];
22  assign PC_Plus_8D = PC_Plus_4F;
23  assign CondD = InstrD[31:28];
24  assign shControlD = InstrD[11:5];
25  RegisterFile RF1 (
26  .CLK(CLK),
27  .WE3(RegWriteW),
28  .A1(RA1D),
29  .A2(RA2D),
30  .A3(WA3W),
31  .WD3(ResultW),
32  .R15(PC_Plus_8D),
33
34  .RD1(RD1D),
35  .RD2(RD2D)
36  );
37  assign InstrImmD = InstrD[23:0];
38
39  Extend Extend1 (
40  .ImmSrc(ImmSrcD),
41  .InstrImm(InstrImmD),
42
43  .ExtImm(ExtImmD)
44  );
45
46  // PIPLINE 3
47  always @(posedge CLK) begin
48  if (FlushE) begin
49  InstrE <= 0;
50
51  PCSE <= 0;
52  RegWE <= 0;
53  MemWE <= 0;
54  FlagWE <= 0;
55  ALUControlE <= 0;
56  MemtoRegE <= 0;
57  ALUSrcE <= 0;
58  NoWriteE <= 0;
59  M_StartE <= 0;
60  MCycleOpE <= 0;
61  M_WE <= 0;
62
```

```
63    CondE <= 0;
64    shControlE <= 0;
65
66    RA1E <= 0;
67    RA2E <= 0;
68    WA3E <= 0;
69    RD1E <= 0;
70    RD2E <= 0;
71    ExtImmE <= 0;
72    end
73    else if (StallE) begin
74    InstrE <= InstrE;
75
76    PCSE <= PCSE;
77    RegWE <= RegWE;
78    MemWE <= MemWE;
79    FlagWE <= FlagWE;
80    ALUControlE <= ALUControlE;
81    MemtoRegE <= MemtoRegE;
82    ALUSrcE <= ALUSrcE;
83    NoWriteE <= NoWriteE;
84    M_StartE <= M_StartE;
85    MCycleOpE <= MCycleOpE;
86    M_WE <= M_WE;
87
88    CondE <= CondE;
89    shControlE <= shControlE;
90
91    RA1E <= RA1E;
92    RA2E <= RA2E;
93    WA3E <= WA3E;
94    RD1E <= RD1E;
95    RD2E <= RD2E;
96    ExtImmE <= ExtImmE;
97    end
98    else begin
99    InstrE <= InstrD;
100
101   PCSE <= PCSD;
102   RegWE <= RegWD;
103   MemWE <= MemWD;
104   FlagWE <= FlagWD;
105   ALUControlE <= ALUControlD;
106   MemtoRegE <= MemtoRegD;
107   ALUSrcE <= ALUSrcD;
108   NoWriteE <= NoWriteD;
109   M_StartE <= M_StartD;
```

```
110   MCycleOpE <= MCycleOpD;
111   M_WE <= M_WD;
112
113   CondE <= CondD;
114   shControlE <= shControlD;
115
116   RA1E <= RA1D;
117   RA2E <= RA2D;
118   WA3E <= WA3D;
119   RD1E <= RD1D;
120   RD2E <= RD2D;
121   ExtImmE <= ExtImmD;
122   end
123   end
124
125   CondLogic CondLogic1(
126   .CLK(CLK),
127   .PCS(PCSE),
128   .RegW(RegWE),
129   .MemW(MemWE),
130   .NoWrite(NoWriteE),
131   .FlagW(FlagWE),
132   .Cond(CondE),
133   .ALUFlags(ALUFlagsE),
134   .M_W(M_WE),
135
136   .PCSrc(PCSrcE),
137   .RegWrite(RegWriteE),
138   .MemWrite(MemWriteE),
139   .M_Write(M_WriteE)
140   );
141
142   assign ShE = shControlE[1:0];
143   assign Shamt5E = shControlE[6:2];
144   assign ShInE = ForwardBE[1]? OpResultM: (ForwardBE[0]? ResultW: RD2E);
145   assign WriteDataE = ShInE;
146
147   Shifter Shifter1(
148   .Sh(ShE),
149   .Shamt5(Shamt5E),
150   .ShIn(ShInE),
151
152   .ShOut(ShOutE)
153   );
154
155   assign SrcAE = ForwardAE[1]? OpResultM: (ForwardAE[0]? ResultW: RD1E);
156   assign SrcBE = ALUSrcE? ExtImmE: ShOutE;
```

```
157
158   ALU ALU1 (
159   .Src_A(SrcAE),
160   .Src_B(SrcBE),
161   .ALUControl(ALUControlE),
162
163   .ALUResult(ALUResultE),
164   .ALUFlags(ALUFlagsE)
165   );
166
167   MCycle MCycle1 (
168   .CLK(CLK),
169   .RESET(Reset),
170   .Start(M_StartE),
171   .MCycleOp(MCycleOpE),
172   .Operand1(SrcAE),
173   .Operand2(WriteDataE),
174
175   .Result(MCycleResultE),
176   .Busy(M_BusyE)
177   );
178
179   assign OpResultE = M_WriteE? MCycleResultE: ALUResultE;
180
181   // PIPLINE 4
182   always @(posedge CLK) begin
183   if (FlushM) begin
184   InstrM <= 0;
185
186   RegWriteM <= 0;
187   MemWriteM <= 0;
188   MemtoRegM <= 0;
189   OpResultM <= 0;
190   WriteDataM <= 0;
191   RA2M <= 0;
192   WA3M <= 0;
193   end
194   else begin
195   InstrM <= InstrE;
196
197   RegWriteM <= RegWriteE;
198   MemWriteM <= MemWriteE;
199   MemtoRegM <= MemtoRegE;
200   OpResultM <= OpResultE;
201   WriteDataM <= WriteDataE;
202   RA2M <= RA2E;
203   WA3M <= WA3E;
```

```
204   end
205   end
206
207   assign MemWrite = MemWriteM;
208   assign OpResult = OpResultM;
209   assign WriteData = ForwardM? ResultW: WriteDataM;
210   assign ReadDataM = ReadData;
211
212   // PIPLINE 5
213   always @(posedge CLK) begin
214   InstrW <= InstrM;
215
216   RegWriteW <= RegWriteM;
217   MemtoRegW <= MemtoRegM;
218   ReadDataW <= ReadDataM;
219   OpResultW <= OpResultM;
220   WA3W <= WA3M;
221   end
222   assign ResultW = MemtoRegW? ReadDataW: OpResultW;
223
224   HazardUnit HazardUnit1 (
225   .RA1D(RA1D),
226   .RA2D(RA2D),
227   .RA1E(RA1E),
228   .RA2E(RA2E),
229   .WA3E(WA3E),
230   .RegWriteE(RegWriteE),
231   .MemtoRegE(MemtoRegE),
232   .PCSrcE(PCSrcE),
233   .M_BusyE(M_BusyE),
234   .WA3M(WA3M),
235   .RegWriteM(RegWriteM),
236   .RA2M(RA2M),
237   .MemWriteM(MemWriteM),
238   .WA3W(WA3W),
239   .MemtoRegW(MemtoRegW),
240   .RegWriteW(RegWriteW),
241
242   .StallF(StallF),
243   .StallD(StallD),
244   .FlushD(FlushD),
245   .StallE(StallE),
246   .FlushE(FlushE),
247   .ForwardAE(ForwardAE),
248   .ForwardBE(ForwardBE),
249   .FlushM(FlushM),
250   .ForwardM(ForwardM)
```

```
);

Next, we will present the crucial Hazard_Unit module.

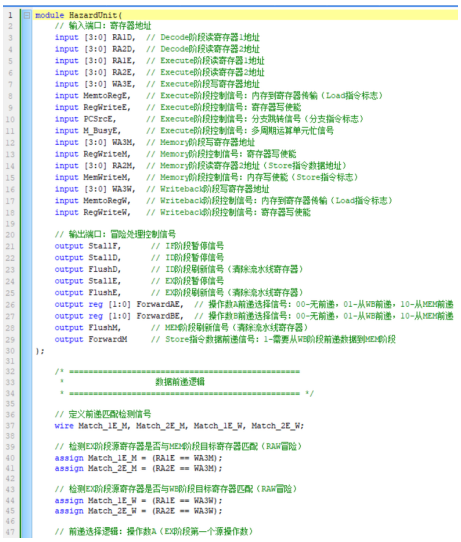
module HazardUnit(
input [3:0] RA1D,
input [3:0] RA2D,
input [3:0] RA1E,
input [3:0] RA2E,
input [3:0] WA3E,
input MemtoRegE,
input RegWriteE,
input PCSrcE,
input M_BusyE,
input [3:0] WA3M,
input RegWriteM,
input [3:0] RA2M,
input MemWriteM,
input [3:0] WA3W,
input MemtoRegW,
input RegWriteW,

output StallF,
output StallD,
output FlushD,
output StallE,
output FlushE,
output reg [1:0] ForwardAE,
output reg [1:0] ForwardBE,
output FlushM,
output ForwardM
);

// Data forwarding for DP
wire Match_1E_M, Match_2E_M, Match_1E_W, Match_2E_W;
assign Match_1E_M = (RA1E == WA3M);
assign Match_2E_M = (RA2E == WA3M);
assign Match_1E_W = (RA1E == WA3W);
assign Match_2E_W = (RA2E == WA3W);
always @(*) begin
if (Match_1E_M && RegWriteM) begin
ForwardAE = 2'b10;
end
else if (Match_1E_W && RegWriteW) begin
ForwardAE = 2'b01;
end
else begin
```

```
298   ForwardAE = 2'b00;
299   end
300   end
301   always @(*) begin
302   if (Match_2E_M && RegWriteM) begin
303   ForwardBE = 2'b10;
304   end
305   else if (Match_2E_W && RegWriteW) begin
306   ForwardBE = 2'b01;
307   end
308   else begin
309   ForwardBE = 2'b00;
310   end
311   end
312
313   // Data forwarding for Mem
314   assign ForwardM = (RA2M == WA3M) & MemWriteM & MemtoRegW & RegWriteM;
315
316
317   // Stalling for Load and Use
318   wire Match_12D_E;
319   assign Match_12D_E = (RA1D == WA3E) | (RA2D == WA3E);
320   wire Idrstall;
321   assign Idrstall = Match_12D_E & MemtoRegE & RegWriteE;
322   // Stalling for Branch
323   wire BranchStall;
324   assign BranchStall = PCSrcE;
325   // Stalling for MCycle
326   wire MCycleStall;
327   assign MCycleStall = M_BusyE;
328
329   assign StallF = Idrstall || MCycleStall;
330   assign StallD = Idrstall || MCycleStall;
331   assign StallE = MCycleStall;
332   assign FlushD = BranchStall;
333   assign FlushE = Idrstall || BranchStall;
334   assign FlushM = MCycleStall;
335   endmodule
```

Detailed code explanations will be provided in the form of comments and presented in the form of pictures. Otherwise, the layout would not be so clear and understandable.



```
module HazardUnit(
    // 输入端口：寄存器地址
    input [3:0] RA1D,  // Decode阶段读寄存器1地址
    input [3:0] RA2D,  // Decode阶段读寄存器2地址
    input [3:0] RA1E,  // Execute阶段读寄存器1地址
    input [3:0] RA2E,  // Execute阶段读寄存器2地址
    input [3:0] WA3E,  // Execute阶段写寄存器地址
    input MemtoRegE,   // Execute阶段控制信号：内存到寄存器传输（Load指令标志）
    input RegWriteE,   // Execute阶段控制信号：寄存器写使能
    input PCSrcE,      // Execute阶段控制信号：分支跳转信号（分支指令标志）
    input M_BusyE,     // Execute阶段控制信号：多周期运算单元忙信号
    input [3:0] WA3M,  // Memory阶段写寄存器地址
    input RegWriteM,   // Memory阶段控制信号：寄存器写使能
    input [3:0] RA2M,  // Memory阶段读寄存器2地址（Store指令数据地址）
    input MemWriteM,   // Memory阶段控制信号：内存写使能（Store指令标志）
    input [3:0] WA3W,  // Writeback阶段写寄存器地址
    input MemtoRegW,   // Writeback阶段控制信号：内存到寄存器传输（Load指令标志）
    input RegWriteW,   // Writeback阶段控制信号：寄存器写使能

    // 输出端口：冒险处理控制信号
    output StallF,     // IF阶段暂停信号
    output StallD,     // ID阶段暂停信号
    output FlushD,     // ID阶段刷新信号（清标流水线寄存器）
    output StallE,     // EX阶段暂停信号
    output FlushE,     // EX阶段刷新信号（清标流水线寄存器）
    output reg [1:0] ForwardAE,  // 操作数A前递选择信号：00-无前递，01-从WB前递，10-从MEM前递
    output reg [1:0] ForwardBE,  // 操作数B前递选择信号：00-无前递，01-从WB前递，10-从MEM前递
    output FlushM,     // MEM阶段刷新信号（清标流水线寄存器）
    output ForwardM    // Store指令数据前递信号：1-需要从WB阶段前递数据到MEM阶段
);

    /* ==========================================
     *                数据前递逻辑
     * ========================================== */
    // 定义前递匹配检测信号
    wire Match_1E_M, Match_2E_M, Match_1E_W, Match_2E_W;

    // 检测EX阶段源寄存器是否与MEM阶段目标寄存器匹配（RAW冒险）
    assign Match_1E_M = (RA1E == WA3M);
    assign Match_2E_M = (RA2E == WA3M);

    // 检测EX阶段源寄存器是否与WB阶段目标寄存器匹配（RAW冒险）
    assign Match_1E_W = (RA1E == WA3W);
    assign Match_2E_W = (RA2E == WA3W);

    // 前递选择逻辑：操作数A（EX阶段第一个源操作数）
```



```
    always @(*) begin
        // 优先级：MEM阶段 > WB阶段 > 无前递
        if (Match_1E_M && RegWriteM) begin
            // 情况1：数据在MEM阶段可用（最近的数据）
            // 将ForwardAE设置为2'b10，表示从MEM阶段前递数据
            ForwardAE = 2'b10;
        end
        else if (Match_1E_W && RegWriteW) begin
            // 情况2：数据在WB阶段可用（较旧的数据）
            // 将ForwardAE设置为2'b01，表示从WB阶段前递数据
            ForwardAE = 2'b01;
        end
        else begin
            // 情况3：无数据冒险，使用原始寄存器文件值
            ForwardAE = 2'b00;
        end
    end

    // 前递选择逻辑：操作数B（EX阶段第二个源操作数）
    always @(*) begin
        // 优先级：MEM阶段 > WB阶段 > 无前递
        if (Match_2E_M && RegWriteM) begin
            // 情况1：数据在MEM阶段可用（最近的数据）
            // 将ForwardBE设置为2'b10，表示从MEM阶段前递数据
            ForwardBE = 2'b10;
        end
        else if (Match_2E_W && RegWriteW) begin
            // 情况2：数据在WB阶段可用（较旧的数据）
            // 将ForwardBE设置为2'b01，表示从WB阶段前递数据
            ForwardBE = 2'b01;
        end
        else begin
            // 情况3：无数据冒险，使用原始寄存器文件值
            ForwardBE = 2'b00;
        end
    end

    // Store指令数据前递检测（Mem-Mem冒险）
    // 当MEM阶段的Store指令需要写入的数据来自之前Load指令时，
    // 需要从WB阶段前递Load指令读取的数据到MEM阶段的Store指令
    assign ForwardM = (RA2M == WA3M) & MemWriteM & MemtoRegW & RegWriteM;
    // 条件解释：
    // 1. (RA2M == WA3M)：MEM阶段Store指令的源寄存器地址等于WB阶段的目标寄存器地址
    // 2. MemWriteM：当前指令是Store指令（需要写内存）
    // 3. MemtoRegW：WB阶段指令是Load指令（从内存读取数据到寄存器）
    // 4. RegWriteM：MEM阶段寄存器写使能有效
```



```
    /* ==========================================
     *              暂停和刷新控制逻辑
     * ========================================== */

    // Load-Use冒险检测：当ID阶段指令依赖于EX阶段的Load指令结果时
    wire Match_12D_E;
    assign Match_12D_E = (RA1D == WA3E) | (RA2D == WA3E);
    // 检查ID阶段的源寄存器是否与EX阶段的目标寄存器匹配

    wire Idrstall;
    assign Idrstall = Match_12D_E & MemtoRegE & RegWriteE;
    // Load-Use冒险发生条件：
    // 1. 寄存器地址匹配（RAW依赖）
    // 2. MemtoRegE=1：EX阶段指令是Load指令（从内存读取）
    // 3. RegWriteE=1：EX阶段指令会写回寄存器

    // 分支冒险检测：当EX阶段执行分支指令时
    wire BranchStall;
    assign BranchStall = PCSrcE;
    // PCSrcE=1表示EX阶段指令是分支指令且需要跳转

    // 多周期指令冒险检测：当EX阶段多周期运算单元忙碌时
    wire MCycleStall;
    assign MCycleStall = M_BusyE;
    // M_BusyE=1表示乘法器/除法器等多周期单元正在工作

    // 流水线暂停控制信号分配
    assign StallF = Idrstall || MCycleStall;  // IF阶段暂停：Load-Use冒险或多周期指令
    assign StallD = Idrstall || MCycleStall;  // ID阶段暂停：Load-Use冒险或多周期指令
    assign StallE = MCycleStall;              // EX阶段暂停：仅多周期指令需要

    // 流水线刷新控制信号分配
    assign FlushD = BranchStall;              // ID阶段刷新：分支跳转时清除错误取指的指令
    assign FlushE = Idrstall || BranchStall;  // EX阶段刷新：Load-Use冒险或分支跳转时
    assign FlushM = MCycleStall;              // MEM阶段刷新：多周期指令可能导致需要清除

    // 信号功能总结：
    // Stall信号：保持当前流水线阶段不变，插入气泡（NOP）
    // Flush信号：清除流水线寄存器内容，防止错误指令传播

    // 冒险处理优先级：
    // 1. Load-Use冒险：最高优先级，需要暂停流水线
    // 2. 分支冒险：中等优先级，可能需要刷新错误路径指令
    // 3. 多周期冒险：最低优先级，根据指令类型处理

endmodule
```

Then comes the RF module. The main modification is the use of different clock edges:

```
// declare RegBank
reg [31:0] RegBank[0:14] ;

// Read Operation: Combinational Logical
assign RD1 = (A1 != 4'd15)? RegBank[A1]: R15;
assign RD2 = (A2 != 4'd15)? RegBank[A2]: R15;

// Write Operation: Sequential Logical
always @(negedge CLK) begin
    if (WE3) begin
        RegBank[A3] <= WD3;
    end
    else begin
        RegBank[A3] <= RegBank[A3];
    end
end
```

The modifications for the PC and the transmission of WA3 are all provided in the above code. Thus, the main code content has been described.

1.5 Test code and simulation waveforms.

We used the same wrapper_tb in lab2 and the instructions in the wrapper as follows:

```
`timescale 1ns / 1ps

module Wrapper_tb;
reg [6:0] DIP;
reg RESET,CLK;
wire [15:0] LED;
wire [31:0] SEVENSEGHEX;
Wrapper wrapper1 (DIP,LED,SEVENSEGHEX,RESET,CLK);
initial begin
  DIP = 6'b001100;
  CLK = 0;
  RESET = 0;
  #10
  RESET = 1;
  #10
  RESET = 0;
  #8000
  $finish;
end
always #5 CLK = ~CLK;

initial begin
    $dumpfile("testbench.wave");
    $dumpvars;
    $display("save to testbench.wave");
end
endmodule
```

代码块

```
//----------------------------------------------------------------
// Instruction Memory
//----------------------------------------------------------------
initial begin
INSTR_MEM[0] = 32'hE59F1204;
INSTR_MEM[1] = 32'hE59F2204;
INSTR_MEM[2] = 32'hE59F31F0;
INSTR_MEM[3] = 32'hE59F41F0;
INSTR_MEM[4] = 32'hE0815002;
INSTR_MEM[5] = 32'hE5835004;
INSTR_MEM[6] = 32'hE2833008;
INSTR_MEM[7] = 32'hE5135004;
INSTR_MEM[8] = 32'hE0416002;
INSTR_MEM[9] = 32'hE5046004;
INSTR_MEM[10] = 32'hE2444008;
INSTR_MEM[11] = 32'hE5946004;
INSTR_MEM[12] = 32'hE2013000;
INSTR_MEM[13] = 32'hE59FA1D8;
INSTR_MEM[14] = 32'hE181400A;
INSTR_MEM[15] = 32'hE1811004;
INSTR_MEM[16] = 32'hE0022003;
INSTR_MEM[17] = 32'hE0017801;
INSTR_MEM[18] = 32'hE0018821;
INSTR_MEM[19] = 32'hE0017447;
INSTR_MEM[20] = 32'hE0018848;
INSTR_MEM[21] = 32'hE0889005;
```

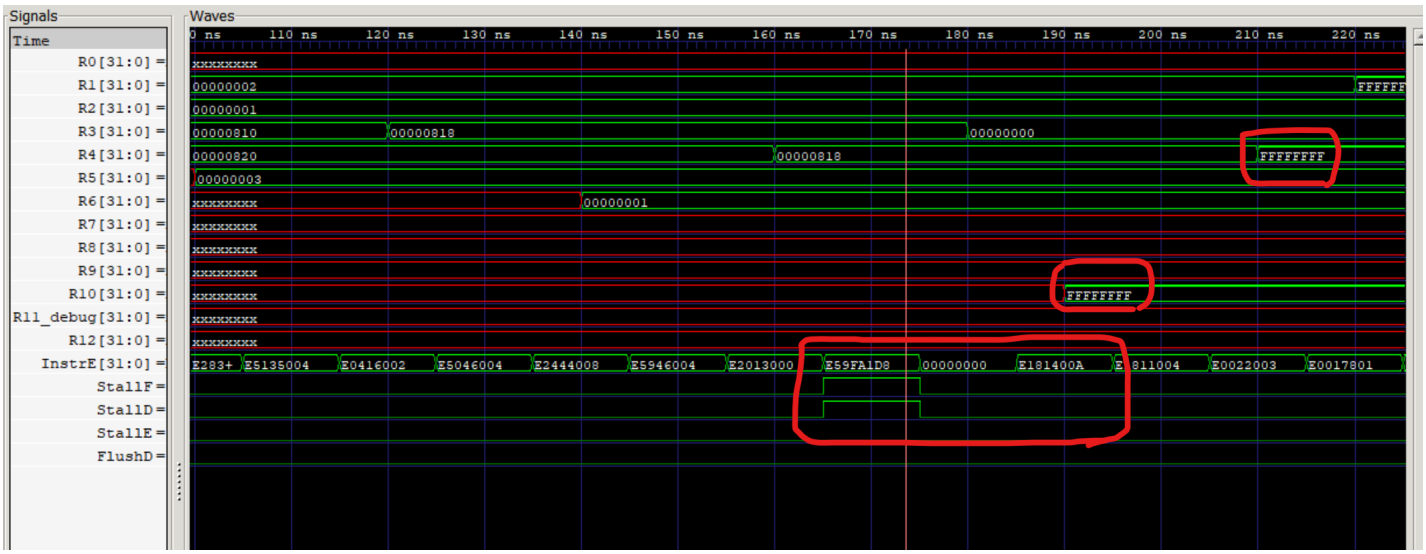
```
27   INSTR_MEM[22] = 32'hE047A006;
28   INSTR_MEM[23] = 32'hE04AB009;
29   INSTR_MEM[24] = 32'hE04BB009;
30   INSTR_MEM[25] = 32'hE001B86B;
31   INSTR_MEM[26] = 32'hE59FC1AC;
32   INSTR_MEM[27] = 32'hE18CB86B;
33   INSTR_MEM[28] = 32'hE59FA1A0;
34   INSTR_MEM[29] = 32'hE04AB00B;
35   INSTR_MEM[30] = 32'hE28BB003;
36   INSTR_MEM[31] = 32'hE28B3002;
37   INSTR_MEM[32] = 32'hE59F1194;
38   INSTR_MEM[33] = 32'hE59F2190;
39   INSTR_MEM[34] = 32'hE24BB001;
40   INSTR_MEM[35] = 32'hE2811001;
41   INSTR_MEM[36] = 32'hE35B0002;
42   INSTR_MEM[37] = 32'h1AFFFFFB;
43   INSTR_MEM[38] = 32'hE2433001;
44   INSTR_MEM[39] = 32'hE2822002;
45   INSTR_MEM[40] = 32'hE3530002;
46   INSTR_MEM[41] = 32'h1AFFFFFB;
47   INSTR_MEM[42] = 32'hE2822006;
48   INSTR_MEM[43] = 32'hE081B002;
49   INSTR_MEM[44] = 32'hE38BB001;
50   INSTR_MEM[45] = 32'hE59F1158;
51   INSTR_MEM[46] = 32'hE001B26B;
52   INSTR_MEM[47] = 32'hE59F1150;
53   INSTR_MEM[48] = 32'hE59F2144;
54   INSTR_MEM[49] = 32'hE59F3144;
55   INSTR_MEM[50] = 32'hE082B24B;
56   INSTR_MEM[51] = 32'hE082B22B;
57   INSTR_MEM[52] = 32'hE082B20B;
58   INSTR_MEM[53] = 32'hE082B26B;
59   INSTR_MEM[54] = 32'hE00BB221;
60   INSTR_MEM[55] = 32'hE183B20B;
61   INSTR_MEM[56] = 32'hE18BB44B;
62   INSTR_MEM[57] = 32'hE59FA11C;
63   INSTR_MEM[58] = 32'hE58AB000;
64   INSTR_MEM[59] = 32'hE59F9110;
65   INSTR_MEM[60] = 32'hE5891000;
66   INSTR_MEM[61] = 32'hEAFFFFFE;
67   for(i = 62; i < 128; i = i+1) begin
68   INSTR_MEM[i] = 32'h0;
69   end
70   end
71   //-----------------------------------------------------------------
72   // Data (Constant) Memory
73   //-----------------------------------------------------------------
```

```
74   initial begin
75   DATA_CONST_MEM[0] = 32'h00000810;
76   DATA_CONST_MEM[1] = 32'h00000820;
77   DATA_CONST_MEM[2] = 32'h00000830;
78   DATA_CONST_MEM[3] = 32'h00000002;
79   DATA_CONST_MEM[4] = 32'h00000001;
80   DATA_CONST_MEM[5] = 32'hFFFFFFFF;
81   DATA_CONST_MEM[6] = 32'hFFFFFEFA;
82   for(i = 7; i < 128; i = i+1) begin
83   DATA_CONST_MEM[i] = 32'h0;
84   end
85   end
```

Case of Load and Use hazard:

```
// 地址 0x00000034: LDR R10, [PC, #0x1D8]   ; 加载数据到R10 (PC+8+0x1D8)
INSTR_MEM[13] = 32'hE59FA1D8;

// 地址 0x00000038: ORR R4, R1, R10         ; R4 = R1 | R10
INSTR_MEM[14] = 32'hE181400A;
```



As can be observed in the figure, STALLF and STALLD have successfully performed their functions, pausing for one cycle and eliminating the Load-use risk. R10 and R4 have both obtained the correct values.

Case of Control hazards:

```
// 地址 0x00000088: SUB R11, R11, #1        ; R11 = R11 - 1
INSTR_MEM[34] = 32'hE24BB001;

// 地址 0x0000008C: ADD R1, R1, #1          ; R1 = R1 + 1
INSTR_MEM[35] = 32'hE2811001;

// 地址 0x00000090: CMP R11, #2             ; 比较R11和2（设置标志位）
INSTR_MEM[36] = 32'hE35B0002;

// 地址 0x00000094: BNE 0x00000090          ; 如果R11 != 2，跳转到0x00000090（循环）
INSTR_MEM[37] = 32'h1AFFFFFB;
```



We adopt the default non-jump approach, and if the judgment is incorrect, we use the FLUSHD, E's early BTA method to handle the control risk. As shown in the figure, we have successfully achieved the default non-jump, and when encountering a branch, we clear D and E and then jump.

Case of Data forwarding from M stage to E stage：

```
// 地址 0x00000018: ADD R3, R3, #8          ; R3 = R3 + 8
INSTR_MEM[6] = 32'hE2833008;

// 地址 0x0000001C: LDR R5, [R3, #-4]       ; 从[R3-4]加载到R5
INSTR_MEM[7] = 32'hE5135004;
```



ForwardAE is 10, and R5 remains unchanged at 00000003, indicating a successful forward movement from M to E.

Case of Data forwarding from W stage to E stage：

```
// 地址 0x00000034: LDR R10, [PC, #0x1D8]   ; 加载数据到R10 (PC+8+0x1D8)
INSTR_MEM[13] = 32'hE59FA1D8;

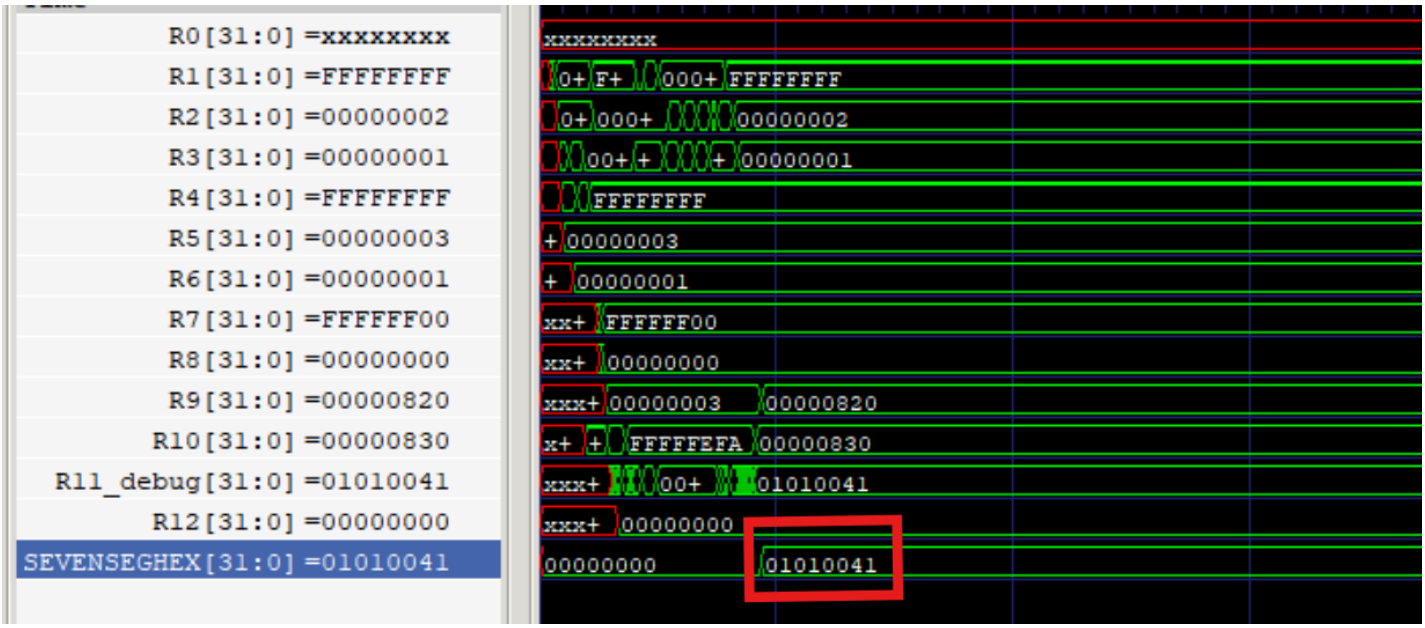
// 地址 0x00000038: ORR R4, R1, R10         ; R4 = R1 | R10
INSTR_MEM[14] = 32'hE181400A;
```



This example can be continued with the "Load-use" example. After the "Load-use" operation completes the "Stall" phase, it still needs to perform the forwarding from the WB to

the EX stage. As can be seen in the diagram, ForwardBE becomes 01, indicating that the forwarding from the WB to the EX stage has been achieved.

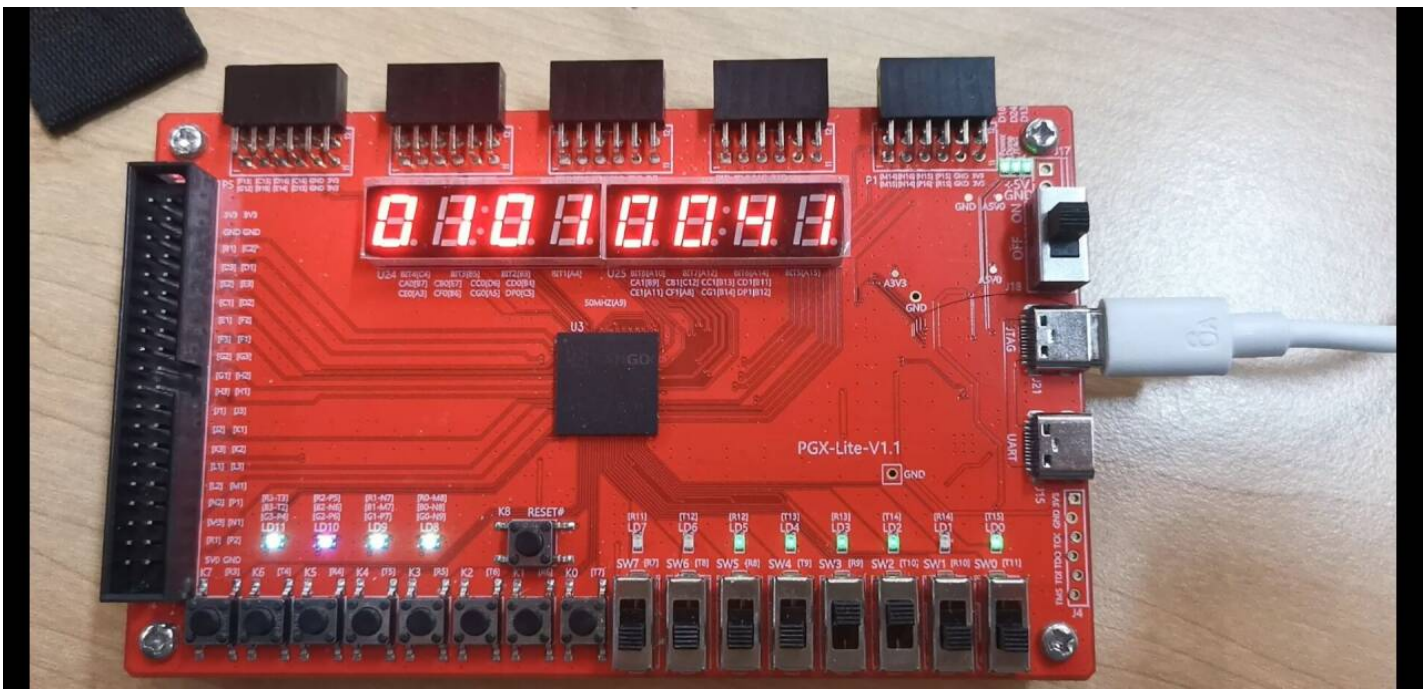
Case of w to m, No data dependency and With data dependency:



The instructions in the LAB do not involve the forward movement from W to M, so we will not discuss them. As for "No data dependency" and "With data dependency", if the program can run and produce results normally, it indicates that they have been handled appropriately.

1.6 Onboard test result

The upper board can obtain 01010041, which is consistent with the waveform diagram, indicating that the final 5-stage pipeline ARM design has been successfully completed.



2. Design and Benchmarking of a RISC-V Processor

2.1 Introduction

2.1.1 Concise Overview of RISC-V ISA (RV32I Focus)

RISC-V is a contemporary instruction set architecture (ISA) that originated at the University of California, Berkeley in 2010. It represents the fifth ISA project undertaken by the university. RISC, which stands for "Reduced Instruction Set Computer," is globally recognized for its superior performance, functional flexibility, and open-source nature. The primary objective of RISC-V is to provide a streamlined and efficient instruction set to enhance processor execution speed and optimize power consumption. By adopting the principles of reduced instruction set design, RISC-V minimizes instruction complexity, thereby improving processor operational efficiency and enabling lower power consumption alongside higher performance.

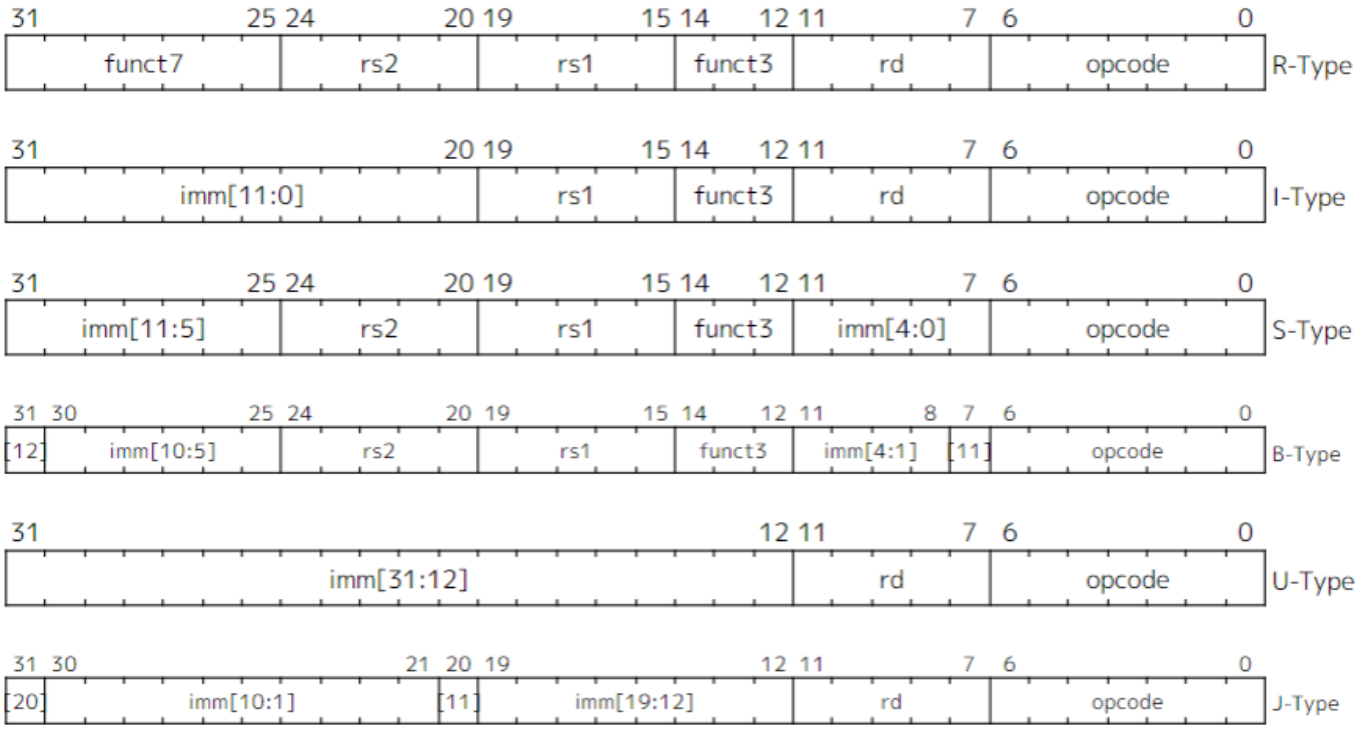
A key characteristic of RISC-V is its scalability. It offers a range of instruction set configurations, including a base instruction set, extended instruction sets, and customizable instruction sets. This diversity allows RISC-V processors to be tailored to meet specific application requirements. Furthermore, RISC-V supports multiple programming languages and software stacks, such as Linux, Android, and BSD, providing developers with extensive options. Additionally, RISC-V employs a modular design approach, starting with a compact base instruction set of only 47 instructions and incorporating various extensions to address diverse application needs.

In terms of performance, RISC-V processors exhibit exceptional instruction execution speed and energy efficiency. The simplified instruction set and highly parallel execution capabilities enable RISC-V processors to execute a large number of instructions rapidly. Moreover, the open-source nature of RISC-V fosters innovation and optimization in processor design, as developers have the freedom to study and enhance processor architectures. There are six instruction types in RISC-V, which are:

- **R-type**: R-type instructions are mainly used to perform arithmetic operations between registers, including addition, subtraction, bit operations, etc.
- **I-type**: type I instructions are used to perform arithmetic operations between registers and immediate numbers, or to read and write to memory.
- **S-type**: S-type instructions are used for memory write operations, that is, storing data taken from a register into memory.
- **B-type**: type B instructions are used to control the flow and mainly include conditional and unconditional branches. These instructions use condition codes such as equal, not equal, greater than, less than, and so on to decide whether to jump or not.
- **U-type**: U-type instructions are usually used for unsigned immediate number operations, mainly for control flow and data transfer.

- **J-type**: J-type instructions are used for jumps, including unconditional jumps and indirect jumps.

  The format of each instruction can be summarized in the following figure:



  The function and controller of all the instructions are described as follows:

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 | rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 | imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srli	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 >= rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 >= rs2) PC += imm	zero-extends
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	
ecall	Environment Call	I	1110011	0x0	imm=0x0	Transfer control to OS	
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Transfer control to debugger	

2.1.2 Key Differences Between RISC-V (RV32I) and ARM

Dimension	RISC-V (RV32I) Architecture	ARM Architecture
Hardware Design	Minimal base hardware requirements (only core modules for RV32I need to be implemented); Open design supports custom extensions (e.g., adding application-specific instructions without changing the base ISA); No mandatory built-in modules (e.g., MMU is optional).	Fixed core variants (e.g., Cortex-A/R/M) with rigid hardware specifications; Closed design with strict extension rules; High-end cores (e.g., Cortex-A) have built-in MMU and other modules, resulting in higher hardware complexity.
Instruction Encoding	RV32I uses 32-bit fixed-length encoding uniformly;	Traditional ARM (ARMv7) supports dual-mode encoding:

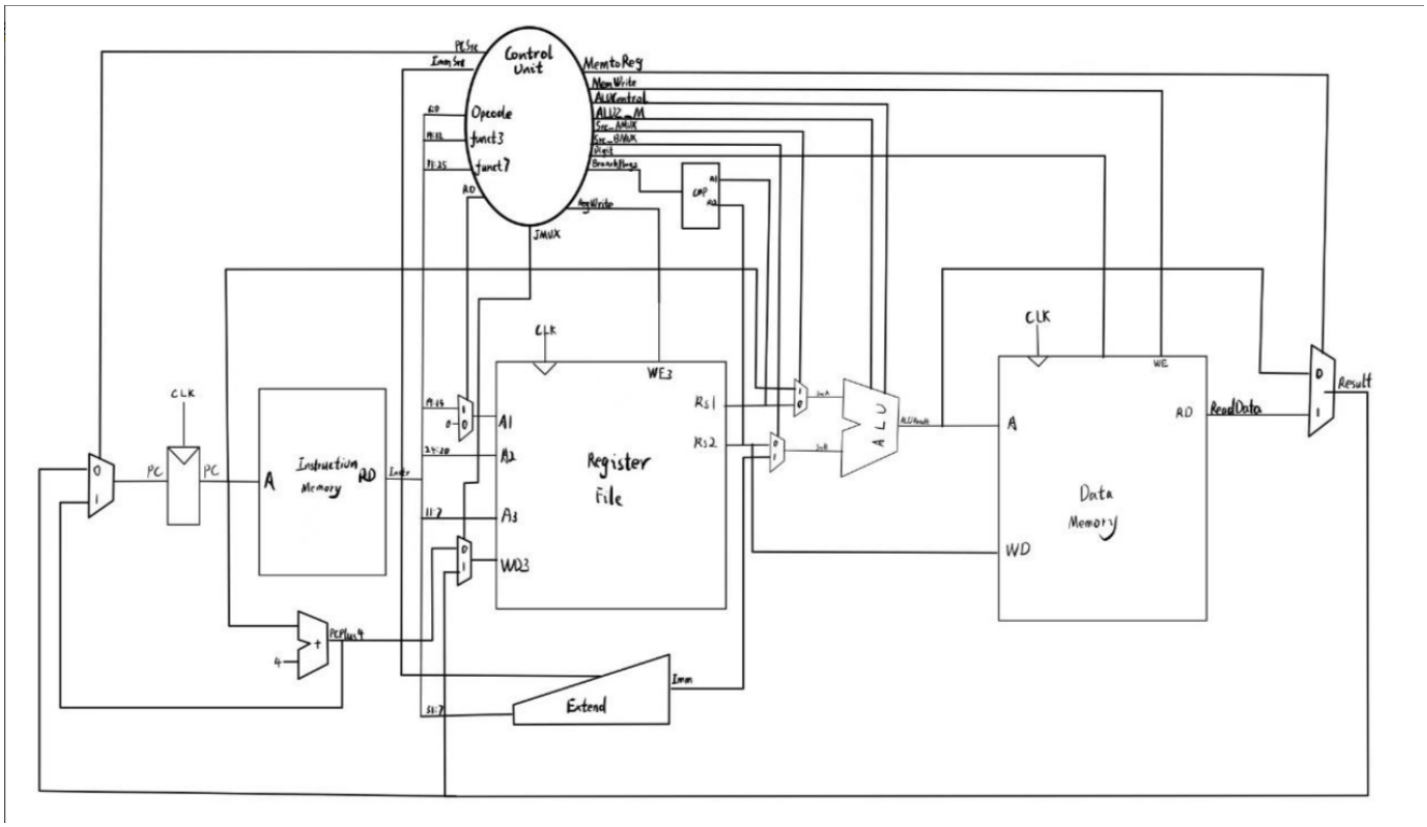
	Compressed 16-bit instructions are supported via the optional RV32C extension (compatible with 32-bit encoding, no separate execution mode required).	32-bit (ARM mode) and 16-bit (Thumb mode); ARMv8-A supports AArch64 (64-bit fixed-length) and AArch32 (backward compatible with ARM/Thumb modes), requiring mode switching logic.
Architectural Philosophy	Openness, minimalism, and extensibility; No backward compatibility constraints, allowing continuous optimization of the base ISA; Designed for diverse scenarios (embedded, server, HPC) with a focus on flexibility.	Closed and proprietary; Prioritizes backward compatibility (supports decades of legacy code), which limits the optimization of new architectures; Initially optimized for mobile/embedded markets, later expanded to servers, with a focus on mature application adaptation.

2.2 Description of CPU Modules and Implementation

2.2.1 Single-Cycle RISC-V (RV32I) CPU Implementation

2.2.1.1 Hardware architecture

The figure below shows the hardware design to implement the RISC-V architecture:



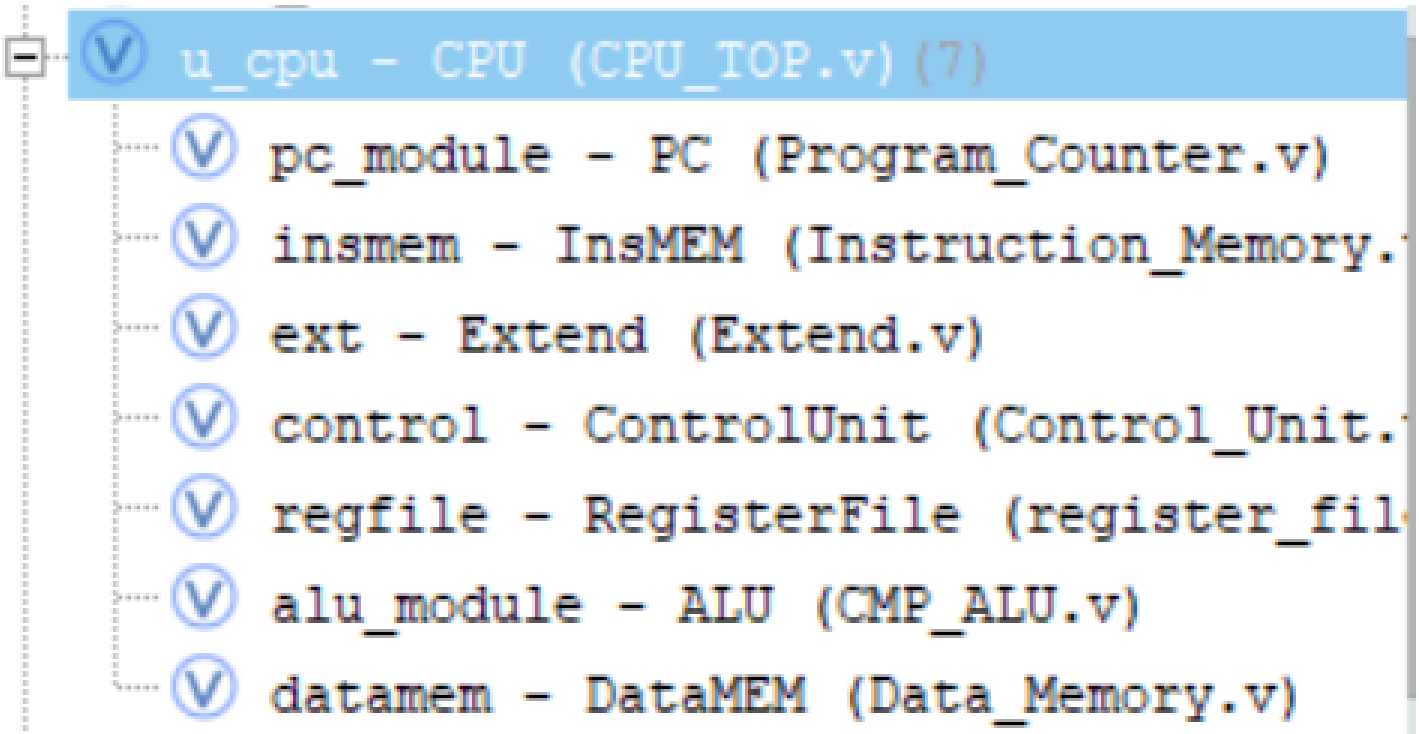
- **Top.cpu** module: The top module, connects each module.
- **Register File**: Accept the address and read and write the corresponding data.
- **Program Counter**: Choose the PC values and control where instructions next.
- **Instruction Memory**: Offer the Instruction at the PC location.
- **Extend**: An extension module to extend an immediate number.
- **Data Memory**: Store the data and get the operation from CPU on the data.
- **Control Unit**: Generating control signals.
- **ALU**: Perform routine mathematical calculations; Choose the right data source; Compare two source data.

2.2.1.2 Top.cpu module

Connect the external DRAM/MMIO interface. Connect each module.

代码块

```
assign perip_addr = AluOutput; // ALU输出作为外设地址
assign perip_wdata = ReadData2; // rs2数据作为写入数据
assign perip_wen = DataWr; // 存储器写使能
assign perip_mask = Digit; // 读写位宽控制
```



2.2.1.3 Control Unit module

12 main control signals (bit width): Reset (1), PCSrc (1), AluOp (3), Alu1Src (1), Alu2Src (1), RegWr (1), RegDst (2), ExtSel (3), Digit (2), DataWr (1), immres (1), Sign (1)

Control signal values and corresponding functions are as follow format:

Signal	Function	jal	jalr	lui	auipc	beq	bne	blt	bge	bltu	bgeu	sb	sh	sw	lb	lh	lw	lbu	lhu
PCSrc	PC next fetch	1	1	0	0	1	1	1	1	1	1	0	0	0	0	0	0	0	0
ALUOp	ALU operation	000	000	X	000	000	000	000	000	000	000	000	000	000	000	000	000	000	000
ALUSrc	ALU1 input	1	0	X	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0
ALU2Src	ALU2 input	1	1	X	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
RegDst	Rd input port	10	10	X	00	X	X	X	X	X	X	X	X	01	01	01	01	01	01
RegWr	Control Rd write enable	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1	1	1	1
ExtSel	Immediate extension mode	100	000	011	011	010	010	010	010	010	010	001	001	001	000	000	000	000	000
Sign	Immediate sign extension	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0
Digit	Control instruction bit number	10	10	10	10	10	10	10	10	10	10	00	01	10	00	01	10	00	01
DataWr	Storage write enable	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0

Signal	Function	add	sub	xor	or	and	sll	srl	sra	slt	sltu	addi	xori	ori	andi	slli	srli	srai	slti	sltiu
PCSrc	PC next fetch	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
ALUOp	ALU operation	000	001	010	011	100	101	110	111	X	X	000	010	011	100	101	110	111	X	X
ALUSrc	ALU1 input	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
ALU2Src	ALU2 input	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
RegDst	Rd input port	00	00	00	00	00	00	00	00	11	11	00	00	00	00	00	00	00	11	11
RegWr	Control Rd write enable	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
ExtSel	Immediate extension mode	X	X	X	X	X	X	X	X	X	X	000	000	000	000	101	101	101	000	000
Sign	Immediate sign extension	X	X	X	X	X	X	X	X	X	X	1	1	1	1	X	X	X	1	0
Digit	Control instruction bit number	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10
DataWr	Storage write enable	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset:

Reset Value	Fuction
0	PC = nextPC
1	PC = 0

PCScr:

PCSrc Value	Fuction
0	nextPC = PC+4
1	nextPC = ALUoutput

AluOp:

AluOp Value	Fuction	AluOp Value	Fuction
000	+	100	&
001	-	101	<<
010	^	110	shift right logical
011	|	111	shift right arithmetic

Alu1Src:

Alu1Src Value	Fuction

0	A = ReadData1 ([rs1])
1	A = PC

Alu2Src：

Alu2Src Value	Fuction
0	A = ReadData2 ([rs2])
1	A = extend (Extend module output)

RegWr：

RegWr Value	Fuction
0	Prohibit writing to the register
1	Allow writing to the register

RegDst：

RegDst Value	Fuction	RegDst Value	Fuction
00	WRdata = ALU module output	10	WRdata = PCoutput
01	WRdata = DataMEM output	11	WRdata = cmp[0]

ExtSel：

ExtSel Value	Fuction	ExtSel Value	Fuction
000	I directive direct extension	010	B Directive extension
101	I Directive interception extension	011	U Directive extension
001	S directive extension	100	J directive extension

Sign：

Sign Value	Fuction

0	Unsigned extension of immediate numbers
1	Immediate numbers have symbolic extensions

Digit:

Digit Value	Fuction
00	The read and write bit width is 8
01	The read and write bit width is 16
10	The read and write bit width is 32

DataWr:

DataWr Value	Fuction
0	Prohibit writing to the memory
1	Allow writing to the memory

immres:

immres Value	Fuction
0	The rd register is written through the RegDst selector.
1	WRdata = extend (The expanded immediate number)

2.2.1.4 PC module

Control the instruction address and achieve the conversion of the curPC and the calculation of the nextPC under the control of the clock pulse.

代码块

```
1       // 检测时钟下降沿计算新指令地址
2       always @(negedge CLK) begin
3           if (Reset) begin
4               nextPC <= 4;
5           end else if (is_load_instr && ~load_stall_reg) begin
6               // 遇到load指令且不在暂停状态，保持nextPC不变
7               nextPC <= nextPC;
```

```
        end else begin
            // 正常计算下一条指令地址
            case(PCSrc)
                1'b0:   nextPC <= curPC + 4;
                1'b1:   nextPC <= AluOutput;
                default: nextPC <= curPC + 4;
            endcase
        end
    end
```

When the load type instruction is detected, pause for one cycle and keep the current PC unchanged to ensure correct reading.

代码块

```
    // 检测是否为load指令（RISC-V中load指令的opcode为7'b0000011）
    wire is_load_instr = (op == 7'b0000011);

        // 暂停状态控制逻辑
    always @(posedge CLK) begin
        if (Reset) begin
            load_stall_reg <= 1'b0;
        end else if (load_stall_reg) begin
            // 如果当前处于暂停状态，下一个周期清除暂停标志
            load_stall_reg <= 1'b0;
        end else begin
            // 检测到load指令时设置暂停标志
            load_stall_reg <= is_load_instr;
        end
    end

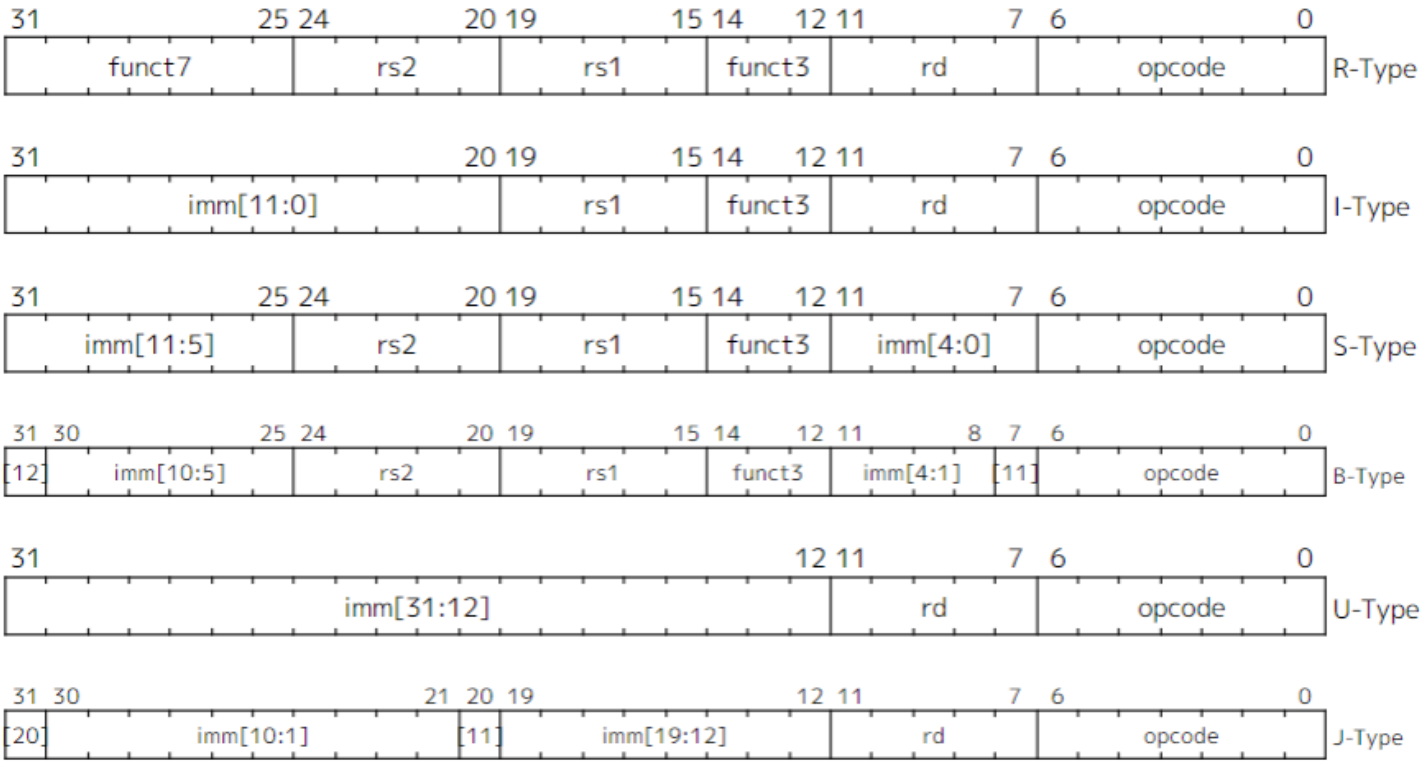
        // 检测时钟上升沿更新PC或复位信号
    always @(posedge CLK or posedge Reset) begin
        if (Reset) begin
            // 复位置1时把PC归零
            curPC <= 0;
        end else if (is_load_instr && ~load_stall_reg) begin
            // 遇到load指令且不在暂停状态，暂停一个周期
            curPC <= curPC;  // PC保持不变
        end else begin
            // 正常执行或暂停后恢复执行
            curPC <= nextPC;
        end
    end
```

2.2.1.5 InsMEM module

According to the instructions obtained from the PC, the instructions are segmented and input to the ControlUnit (op, funct3, funct7), Extend (imm) and RegFile (rd).

代码块

```
    always @(instr) begin
        // 提取指令各字段
        op = {1'b0, instr[6:0]};      // 操作码，补0到8位
        rs1 = instr[19:15];           // 源寄存器1地址
        rs2 = instr[24:20];           // 源寄存器2地址
        rd = instr[11:7];             // 目的寄存器地址
        funct3 = instr[14:12];        // 3位功能码
        funct7 = instr[31:25];        // 7位功能码（修正：应该是[31:25]）
        imm = instr[31:7];            // 立即数字段，供扩展模块使用
    end
```



2.2.1.6 RegFile module

Accept the register address given by the instruction to read and write the corresponding data. Reading data is done in real time by the combinational logic circuit, and writing data is done by the sequential logic circuit at the falling edge of the clock.

Note that when immres is 1, directly use extend as the input data; only when immres is 0 will RegDst be considered.

```
    // 时序逻辑写入：时钟下降沿时更新rd地址的值
    always@(negedge CLK) begin
        // 如果写使能有效且目标寄存器不是x0
        if (RegWr && (WriteReg != 5'b00000)) begin
            if (immres) begin
                // immres为1时，直接写入扩展后的立即数
                regFile[WriteReg] = extend;
                DB = extend;
            end else begin
                // immres为0时，根据RegDst选择写入的数据源
                case (RegDst)
                    2'b00: begin
                        // 从ALU输出写入
                        regFile[WriteReg] = AluOutput;
                        DB = AluOutput;
                    end
                    2'b01: begin
                        // 从数据存储器写入
                        regFile[WriteReg] = perip_rdata;
                        DB = perip_rdata;
                    end
                    2'b10: begin
                        // 写入PC+4（用于JAL，JALR指令）
                        regFile[WriteReg] = PC + 4;
                        DB = PC + 4;
                    end
                    2'b11: begin
                        // 写入比较结果（用于SLT，SLTI，SLTU，SLTIU指令）
                        // cmp[0] = 0: >=，cmp[0] = 1: <
                        regFile[WriteReg] = {31'b0, cmp[0]};
                        DB = {31'b0, cmp[0]};
                    end
                    default: begin
                        // 默认情况，保持原值
//                          regFile[WriteReg] = regFile[WriteReg];
                        DB = 32'b0;
                    end
                endcase
            end
        end else begin
            // 写使能无效或目标寄存器是x0，不写入
            // 不更新DB，保持DB当前值
        end

        // 确保x0寄存器始终为0（防御性编程）
        regFile[0] = 32'd0;
    end
```

2.2.1.7 ALU module

It is equipped with an embedded comparator and an ALU calculator, enabling signed or unsigned comparisons and basic mathematical operations

代码块

```
// 根据指令类型选择正确的比较源
    wire [31:0] actual_operand2;

    // 关键修正：区分R型指令和I型指令的比较源
    assign actual_operand2 = (op == 7'b0010011 &&
                              (funct3 == 3'b010 || funct3 == 3'b011)) ?
                             extend : ReadData2;

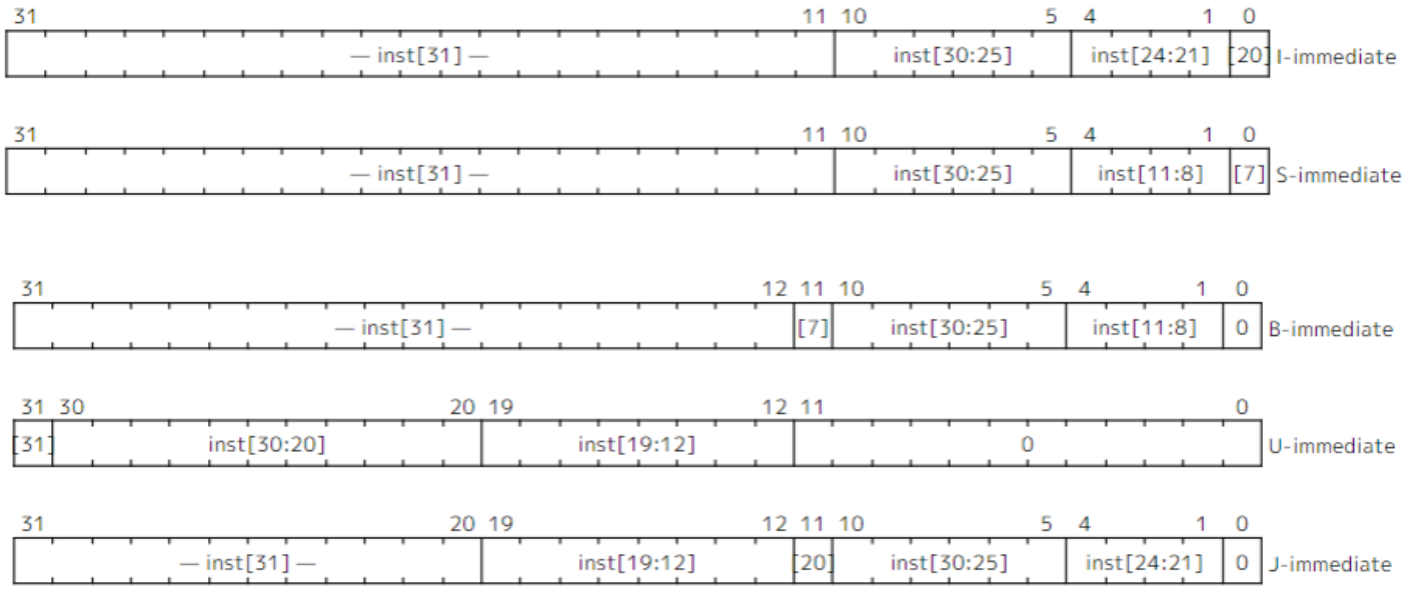
    // 有符号比较逻辑（使用正确的操作数源）
    wire signed_lt = (~ReadData1[31] & actual_operand2[31]) ? 1'b0 :
                     (ReadData1[31] & ~actual_operand2[31]) ? 1'b1 :
                     (ReadData1 < actual_operand2);

    wire unsigned_lt = (ReadData1 < actual_operand2);

            case(AluOp)
            3'b000: AluOutput = A + B;           // ADD
            3'b001: AluOutput = A - B;           // SUB
            3'b010: AluOutput = A ^ B;           // XOR
            3'b011: AluOutput = A | B;           // OR
            3'b100: AluOutput = A & B;           // AND
            3'b101: AluOutput = A << B[4:0];     // SLL
            3'b110: AluOutput = A >> B[4:0];     // SRL
            3'b111:
 begin
    // 算术右移
    if (B[4:0] == 5'b0) begin
        AluOutput = A;
    end else begin
        // 直接使用符号扩展
        AluOutput = (A[31] == 1'b0) ? (A >> B[4:0]) :  // 正数
                    ((A >> B[4:0]) | (~({32{1'b1}} >> B[4:0])));  // 负数
    end
end
            default: AluOutput = 32'b0;
        endcase
```

2.2.1.8 Extend module

Receive the control signal from the ControlUnit to splice and expand the immediate digital segment.



It is worth noting that here, the comparison between signed and unsigned numbers is determined through funct3.

代码块

```
            case(funct3)
                3'b100, 3'b101, 3'b010:  // BLT, BGE, SLT/SLTI
                    cmp = signed_lt ? 2'b01 : 2'b10;
                3'b110, 3'b111, 3'b011:  // BLTU, BGEU, SLTU/SLTIU
                    cmp = unsigned_lt ? 2'b01 : 2'b10;
                default:
                    cmp = signed_lt ? 2'b01 : 2'b10;
            endcase
        end
```

At the same time, the op value is used to accurately distinguish between R-type and I-type instructions based on their comparison source.

代码块

```
    // 关键修正：区分R型指令和I型指令的比较源
    assign actual_operand2 = (op == 7'b0010011 &&
                              (funct3 == 3'b010 || funct3 == 3'b011)) ?
                             extend : ReadData2;
```

2.2.1.9 2.1.9 data memory module

The external part, the cpu receives data through the interface in the cputop section.

代码块

```
1       input  [31:0]  perip_rdata
```

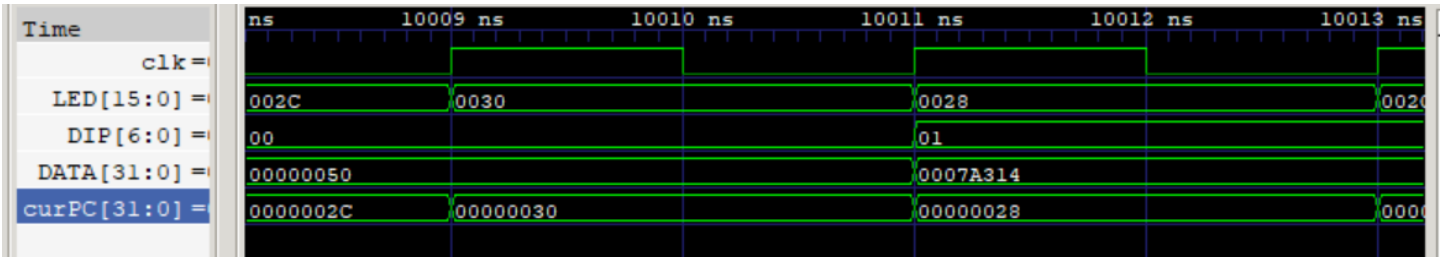
2.2.2 CPU simulation waveforms of single cycle RISC-V (RV32I) CPU

```
0000000000000000 < start>:
   0:  00100293            li      t0,1
   4:  00000313            li      t1,0
   8:  3e800393            li      t2,1000
   c:  90000e37            lui     t3,0x90000
  10:  80000eb7            lui     t4,0x80000
  14:  000eaf03            lw      t5, 0(t4)

0000000000000018 <loop>:
  18:  0053c863            blt     t2,t0,28 <done>
  1c:  00530333            add  t1,t1,t0
  20:  00128293            addi t0,t0,1
  24:  ff5ff06f          j     18 <loop>

0000000000000028 <done>:
  28:  006e2023            sw      t1,0(t3) # ffffffff90000000 <done+0xffffffff8fffffd8>
  2c:  01ce2623            sw      t3,12(t3)
  30:  ff9ff06f          j     28 <done>
```

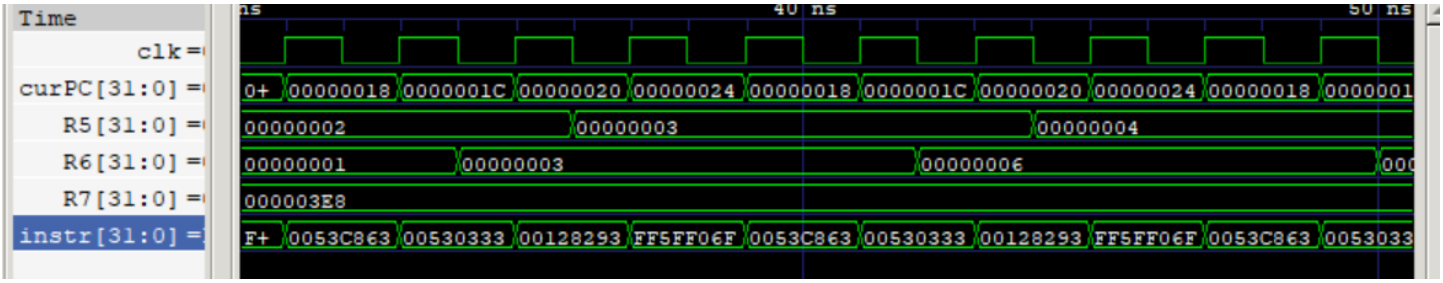
Based on the aabove encoding and decoding, we can see that this is a test where numbers are incremented from 1 to 1000. It involves addition of immediate numbers, jumps, and storage instructions. The final result is 7A314, which will be written into the datamemory. Because the waveform chart is as follows



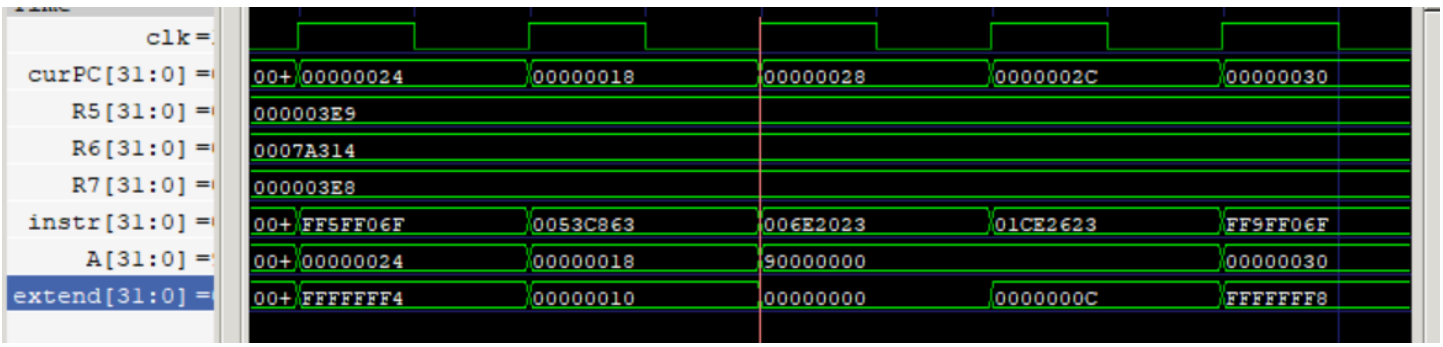
The corresponding registers in the calculation are as follows:

Register	ABI Name	Description
x0	zero	Hard-wired zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5	t0	Temporary/alternate link register
x6–7	t1–2	Temporaries
x8	s0/fp	Saved register/frame pointer
x9	s1	Saved register
x10–11	a0–1	Function arguments/return values
x12–17	a2–7	Function arguments
x18–27	s2–11	Saved registers
x28–31	t3–6	Temporaries

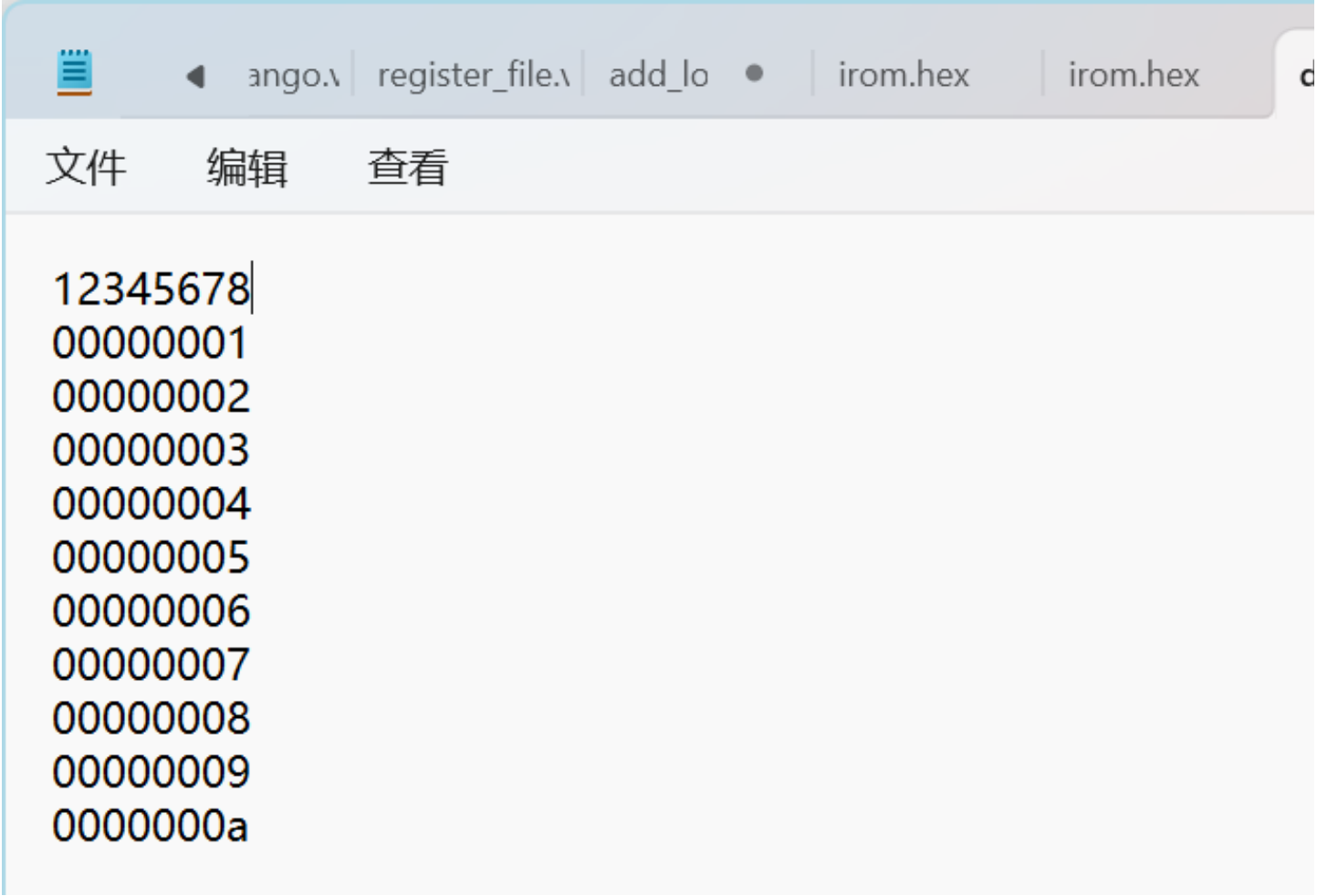
Observe the value of the corresponding register and find that the data has been correctly written into the Register file. The corresponding PC value and instruction are also correct.



When the BLT instruction determines that 1001 is greater than 1000, the PC value correctly jumps to the address calculated in the ALU.



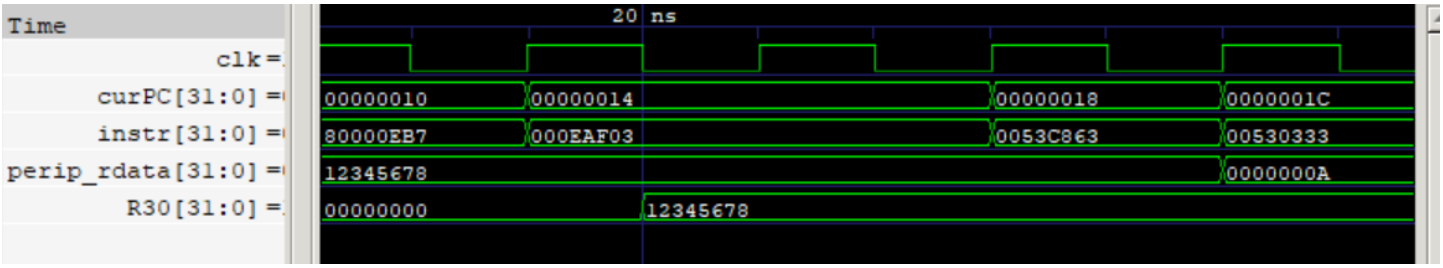
Among them, when the value of pc is 14, it is used for testing the lw instruction. We try to change the dram data as below:



代码块

```
                    3'b010: begin  // LW
                        Digit = 2'b10;
                        Sign = 1;


        assign perip_mask = Digit;        // 读写位宽控制
```



which means perip_rdata will equal 12345678 at lw instruction. The Digit, which is connected to

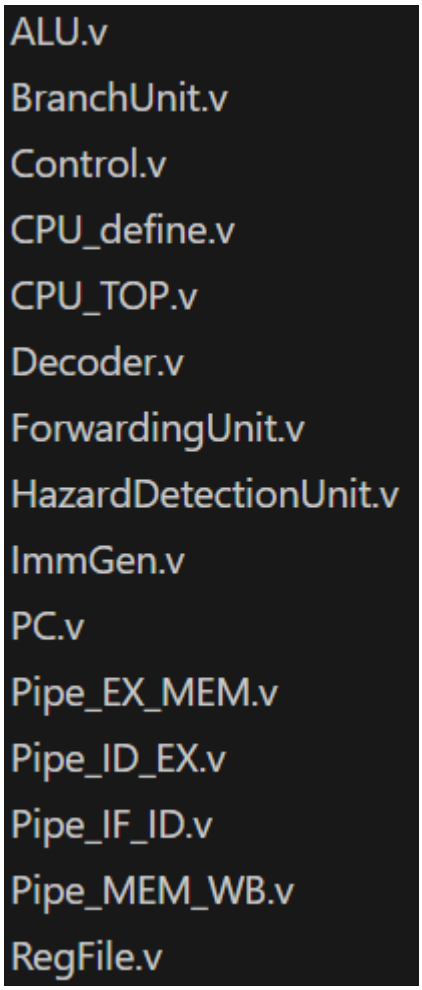
perip_mask, is set 10.

By pausing for one cycle using the PCmodule, the R30 can be read correctly.

2.3 Multi-Cycle RISC-V (RV32I) CPU Implementation(pipeline)

Because the RISC-V single-cycle CPU and multi-cycle CPU code in this report were written separately by two people, there are slight differences in the basic structure of the C modules.

The basic modules of this multi-cycle RISC-V CPU are shown in the figure below.



Based on the ALU, BranchUnit, Control, CPU_TOP, Decoder, ImmGen, PC, and RegFile modules as the basic framework, pipeline modules corresponding to each stage of the five-stage pipeline have been added, along with the ForwardingUnit and HazardDetectionUnit to handle hazards. The following focuses on the modules added to implement the pipeline.

CPU_TOP Module

The `CPU_TOP.v` module serves as the central integration hub for the complete five-stage pipelined RISC-V processor. It orchestrates the interaction between all pipeline stages, hazard resolution units, and external memory interfaces.

代码块

```
1  `include "CPU_define.v"
2
3  module CPU (
4      input         clk,
5      input         rst_n,
6      input         run,
```

```
7
8       // Interface to IROM
9       output [31:0]  pc,
10      input  [31:0]  instruction,
11
12      // Interface to DRAM & MMIO
13      output [31:0]  perip_addr,
14      output [31:0]  perip_wdata,
15      output         perip_wen,
16      output [1:0]   perip_mask,
17      input  [31:0]  perip_rdata
18  );
19
20      // ============================================================================
21      // Internal Signals
22      // ============================================================================
23
24      // Global Stall (Memory Wait)
25      wire global_stall;
26      reg [2:0] mem_wait_cnt;
27
28      // Pipeline control (Generated by Hazard Unit)
29      wire stall_if;
30      wire flush_if_id;
31      wire flush_id_ex;
32
33      // Forwarding (Generated by Forwarding Unit)
34      wire [1:0] forward_a;
35      wire [1:0] forward_b;
36
37      // Wire for Pipeline enable
38      wire pipe_en = run & ~global_stall;
39
40      // ============================================================================
41      // IF Stage
42      // ============================================================================
43      wire [31:0] pc_next;
44      wire [31:0] pc_current;
45
46      // Branch/Jump targets from EX stage
47      wire [31:0] branch_target;
48      wire branch_taken_ex;
49
50      // Allow PC update only if running, no mem stall, and no hazard stall
51      wire pc_en = run & ~global_stall & ~stall_if;
52
53      // Instantiate PC
```

```
54      PC u_pc (
55          .clk(clk),
56          .rst_n(rst_n),
57          .run(pc_en),
58          .pc_next(pc_next),
59          .pc(pc_current)
60      );
61
62      assign pc = pc_current;
63
64      wire [31:0] pc_plus_4 = pc_current + 4;
65
66      // Select next PC: Branch Target if taken, else PC+4
67      assign pc_next = branch_taken_ex ? branch_target : pc_plus_4;
68
69      // ============================================================================
70      // IF/ID Pipeline Register
71      // ============================================================================
72      wire [31:0] if_id_pc;
73      wire [31:0] if_id_inst;
74
75      Pipe_IF_ID u_pipe_if_id (
76          .clk(clk),
77          .rst_n(rst_n),
78          .en(pipe_en),
79          .hold(stall_if),
80          .flush(flush_if_id),
81          .pc_in(pc_current),
82          .inst_in(instruction),
83          .pc_out(if_id_pc),
84          .inst_out(if_id_inst)
85      );
86
87      // ============================================================================
88      // ID Stage
89      // ============================================================================
90      wire [6:0]  id_opcode;
91      wire [4:0]  id_rd, id_rs1, id_rs2;
92      wire [2:0]  id_funct3;
93      wire [6:0]  id_funct7;
94
95      Decoder u_decoder (
96          .instruction(if_id_inst),
97          .opcode(id_opcode),
98          .rd(id_rd),
99          .rs1(id_rs1),
100         .rs2(id_rs2),
```

```
101             .funct3(id_funct3),
102             .funct7(id_funct7)
103         );
104
105         wire [31:0] id_imm;
106         ImmGen u_immgen (
107             .instruction(if_id_inst),
108             .opcode(id_opcode),
109             .imm(id_imm)
110         );
111
112         // Control Unit
113         wire [`ALU_OP_WIDTH-1:0] id_alu_op;
114         wire [1:0] id_src1_sel, id_src2_sel;
115         wire id_mem_wen, id_mem_ren;
116         wire id_reg_wen;
117         wire id_branch_en, id_jump_en;
118         wire [1:0] id_mem_width;
119         wire id_load_unsigned;
120
121         Control u_control (
122             .opcode(id_opcode),
123             .funct3(id_funct3),
124             .funct7(id_funct7),
125             .alu_op(id_alu_op),
126             .src1_sel(id_src1_sel),
127             .src2_sel(id_src2_sel),
128             .mem_wen(id_mem_wen),
129             .mem_ren(id_mem_ren),
130             .reg_wen(id_reg_wen),
131             .branch_en(id_branch_en),
132             .jump_en(id_jump_en),
133             .mem_width(id_mem_width),
134             .load_unsigned(id_load_unsigned)
135         );
136
137         // Register File Interconnections
138         wire [31:0] id_rs1_data_raw, id_rs2_data_raw;
139         wire [31:0] id_rs1_data, id_rs2_data;
140         wire [31:0] wb_rd_data; // From WB stage
141         wire [4:0]  wb_rd_addr; // From WB stage
142         wire        wb_reg_wen; // From WB stage
143
144         RegFile u_regfile (
145             .clk(clk),
146             .rst_n(rst_n),
147             .rs1_addr(id_rs1),
```

```
148          .rs2_addr(id_rs2),
149          .rd_addr(wb_rd_addr),
150          .rd_data(wb_rd_data),
151          .reg_wen(wb_reg_wen & ~global_stall),
152          .rs1_data(id_rs1_data_raw),
153          .rs2_data(id_rs2_data_raw)
154      );
155
156      // WB -> ID Forwarding (Solves dependency for Instruction I+3 reading register written in WB)
157      assign id_rs1_data = (wb_reg_wen && (wb_rd_addr != 5'd0) && (wb_rd_addr == id_rs1)) ? wb_rd_data : id_rs1_data_raw;
158      assign id_rs2_data = (wb_reg_wen && (wb_rd_addr != 5'd0) && (wb_rd_addr == id_rs2)) ? wb_rd_data : id_rs2_data_raw;
159
160      // ============================================================================
161      // ID/EX Pipeline Register
162      // ============================================================================
163      wire [31:0] id_ex_pc;
164      wire [31:0] id_ex_imm;
165      wire [31:0] id_ex_rs1_data;
166      wire [31:0] id_ex_rs2_data;
167      wire [4:0]  id_ex_rd, id_ex_rs1, id_ex_rs2;
168      wire [2:0]  id_ex_funct3;
169
170      wire [`ALU_OP_WIDTH-1:0] id_ex_alu_op;
171      wire [1:0]  id_ex_src1_sel;
172      wire [1:0]  id_ex_src2_sel;
173      wire        id_ex_mem_wen;
174      wire        id_ex_mem_ren;
175      wire        id_ex_reg_wen;
176      wire        id_ex_branch_en;
177      wire        id_ex_jump_en;
178      wire [1:0]  id_ex_mem_width;
179      wire        id_ex_load_unsigned;
180
181      Pipe_ID_EX u_pipe_id_ex (
182          .clk(clk),
183          .rst_n(rst_n),
184          .en(pipe_en),
185          .flush(flush_id_ex),
186          // Inputs
187          .pc_in(if_id_pc),
188          .imm_in(id_imm),
189          .rs1_data_in(id_rs1_data),
190          .rs2_data_in(id_rs2_data),
191          .rd_in(id_rd),
```

```
192             .rs1_in(id_rs1),
193             .rs2_in(id_rs2),
194             .funct3_in(id_funct3),
195             .alu_op_in(id_alu_op),
196             .src1_sel_in(id_src1_sel),
197             .src2_sel_in(id_src2_sel),
198             .mem_wen_in(id_mem_wen),
199             .mem_ren_in(id_mem_ren),
200             .reg_wen_in(id_reg_wen),
201             .branch_en_in(id_branch_en),
202             .jump_en_in(id_jump_en),
203             .mem_width_in(id_mem_width),
204             .load_unsigned_in(id_load_unsigned),
205             // Outputs
206             .pc_out(id_ex_pc),
207             .imm_out(id_ex_imm),
208             .rs1_data_out(id_ex_rs1_data),
209             .rs2_data_out(id_ex_rs2_data),
210             .rd_out(id_ex_rd),
211             .rs1_out(id_ex_rs1),
212             .rs2_out(id_ex_rs2),
213             .funct3_out(id_ex_funct3),
214             .alu_op_out(id_ex_alu_op),
215             .src1_sel_out(id_ex_src1_sel),
216             .src2_sel_out(id_ex_src2_sel),
217             .mem_wen_out(id_ex_mem_wen),
218             .mem_ren_out(id_ex_mem_ren),
219             .reg_wen_out(id_ex_reg_wen),
220             .branch_en_out(id_ex_branch_en),
221             .jump_en_out(id_ex_jump_en),
222             .mem_width_out(id_ex_mem_width),
223             .load_unsigned_out(id_ex_load_unsigned)
224         );
225 
226         // ============================================================================
227         // EX Stage
228         // ============================================================================
229 
230         // Look-ahead declarations for Pipeline Registers
231         wire [31:0] ex_mem_alu_result;
232         wire [4:0]  ex_mem_rd;
233         wire        ex_mem_reg_wen;
234         wire [31:0] ex_mem_pc_plus_4;
235         wire        ex_mem_mem_ren;
236         wire        ex_mem_jump_en;
237 
238         wire [31:0] ex_rs1_final;
```

```
239     wire [31:0] ex_rs2_final;
240 
241     // Generate MEM_ForwardData (Logic from Reference adapted to local signals)
242     // This allows forwarding of Load Results (if timing allows) and Jumps from MEM stage
243     wire [31:0] mem_forward_data;
244     assign mem_forward_data = (ex_mem_mem_ren) ? perip_rdata :        // Forward Load Data
245                               (ex_mem_jump_en) ? ex_mem_pc_plus_4 : // Forward JAL/JALR return address
246                               ex_mem_alu_result;                    // Forward ALU Result (default)
247 
248     assign ex_rs1_final = (forward_a == 2'b01) ? mem_forward_data :
249                           (forward_a == 2'b10) ? wb_rd_data :
250                           id_ex_rs1_data;
251 
252     assign ex_rs2_final = (forward_b == 2'b01) ? mem_forward_data :
253                           (forward_b == 2'b10) ? wb_rd_data :
254                           id_ex_rs2_data;
255 
256     // -- ALU Input Selection --
257     wire [31:0] alu_src1;
258     wire [31:0] alu_src2;
259 
260     assign alu_src1 = (id_ex_src1_sel == `SRC1_REG) ? ex_rs1_final :
261                       (id_ex_src1_sel == `SRC1_PC)  ? id_ex_pc :
262                       (id_ex_src1_sel == `SRC1_X0)  ? 32'b0 : 32'b0;
263 
264     assign alu_src2 = (id_ex_src2_sel == `SRC2_REG) ? ex_rs2_final :
265                       (id_ex_src2_sel == `SRC2_IMM) ? id_ex_imm : 32'b0;
266 
267     // -- ALU Execution --
268     wire [31:0] alu_result;
269     wire zero; // Unused
270 
271     ALU u_alu (
272         .alu_src1(alu_src1),
273         .alu_src2(alu_src2),
274         .alu_op(id_ex_alu_op),
275         .alu_result(alu_result),
276         .zero(zero)
277     );
278 
279     // -- Branch Decision --
280     wire branch_condition_met;
281     BranchUnit u_branch_unit (
```

```
282             .rs1_data(ex_rs1_final),
283             .rs2_data(ex_rs2_final),
284             .funct3(id_ex_funct3),
285             .branch_taken(branch_condition_met)
286         );
287 
288         // Branch/Jump taken signal (Resolves Control Hazard)
289         assign branch_taken_ex = (id_ex_branch_en && branch_condition_met) ||
    id_ex_jump_en;
290 
291         // Target Calculation
292         assign branch_target = (id_ex_jump_en) ? alu_result : (id_ex_pc +
    id_ex_imm);
293 
294         // ============================================================================
295         // EX/MEM Pipeline Register
296         // ============================================================================
297         wire [31:0] ex_mem_rs2_data;
298         wire        ex_mem_mem_wen;
299         wire [1:0]  ex_mem_mem_width;
300         wire        ex_mem_load_unsigned;
301 
302         Pipe_EX_MEM u_pipe_ex_mem (
303             .clk(clk),
304             .rst_n(rst_n),
305             .en(pipe_en),
306             // Inputs
307             .alu_result_in(alu_result),
308             .rs2_data_in(ex_rs2_final),
309             .pc_plus_4_in(id_ex_pc + 32'd4),
310             .rd_in(id_ex_rd),
311             .mem_wen_in(id_ex_mem_wen),
312             .mem_ren_in(id_ex_mem_ren),
313             .reg_wen_in(id_ex_reg_wen),
314             .jump_en_in(id_ex_jump_en),
315             .mem_width_in(id_ex_mem_width),
316             .load_unsigned_in(id_ex_load_unsigned),
317             // Outputs
318             .alu_result_out(ex_mem_alu_result),
319             .rs2_data_out(ex_mem_rs2_data),
320             .pc_plus_4_out(ex_mem_pc_plus_4),
321             .rd_out(ex_mem_rd),
322             .mem_wen_out(ex_mem_mem_wen),
323             .mem_ren_out(ex_mem_mem_ren),
324             .reg_wen_out(ex_mem_reg_wen),
325             .jump_en_out(ex_mem_jump_en),
326             .mem_width_out(ex_mem_mem_width),
```

```
327             .load_unsigned_out(ex_mem_load_unsigned)
328         );
329 
330         // ============================================================================
331         // MEM Stage
332         // ============================================================================
333 
334         // Memory Access
335         assign perip_addr = ex_mem_alu_result;
336         assign perip_wen = ex_mem_mem_wen;
337         assign perip_wdata = ex_mem_rs2_data;
338 
339         // Mask logic
340         assign perip_mask = (ex_mem_mem_width == `MEM_WIDTH_BYTE) ? 2'b00 :
341                             (ex_mem_mem_width == `MEM_WIDTH_HALF) ? 2'b01 :
342                             (ex_mem_mem_width == `MEM_WIDTH_WORD) ? 2'b10 :
343                             2'b10;
344 
345         // Memory Wait State State Machine
346         always @(posedge clk or negedge rst_n) begin
347             if (!rst_n) begin
348                 mem_wait_cnt <= 3'b0;
349             end else if (ex_mem_mem_ren) begin
350                 if (mem_wait_cnt < `__MEM_WAITSTATE__)
351                     mem_wait_cnt <= mem_wait_cnt + 1'b1;
352                 else
353                     mem_wait_cnt <= 3'b0;
354             end else begin
355                 mem_wait_cnt <= 3'b0;
356             end
357         end
358 
359         assign global_stall = (ex_mem_mem_ren && (mem_wait_cnt <
    `__MEM_WAITSTATE__));
360 
361         // ============================================================================
362         // MEM/WB Pipeline Register
363         // ============================================================================
364         wire [31:0] mem_wb_alu_result;
365         wire [31:0] mem_wb_mem_rdata_raw;
366         wire [31:0] mem_wb_pc_plus_4;
367         wire [4:0]  mem_wb_rd;
368         wire        mem_wb_reg_wen;
369         wire        mem_wb_mem_ren;
370         wire        mem_wb_jump_en;
371         wire [1:0]  mem_wb_mem_width;
372         wire        mem_wb_load_unsigned;
```

```
373
374     assign wb_rd_addr = mem_wb_rd;
375     assign wb_reg_wen = mem_wb_reg_wen;
376
377     Pipe_MEM_WB u_pipe_mem_wb (
378         .clk(clk),
379         .rst_n(rst_n),
380         .en(pipe_en),
381         // Inputs
382         .alu_result_in(ex_mem_alu_result),
383         .mem_rdata_in(perip_rdata),
384         .pc_plus_4_in(ex_mem_pc_plus_4),
385         .rd_in(ex_mem_rd),
386         .reg_wen_in(ex_mem_reg_wen),
387         .mem_ren_in(ex_mem_mem_ren),
388         .jump_en_in(ex_mem_jump_en),
389         .mem_width_in(ex_mem_mem_width),
390         .load_unsigned_in(ex_mem_load_unsigned),
391         // Outputs
392         .alu_result_out(mem_wb_alu_result),
393         .mem_rdata_out(mem_wb_mem_rdata_raw),
394         .pc_plus_4_out(mem_wb_pc_plus_4),
395         .rd_out(mem_wb_rd),
396         .reg_wen_out(mem_wb_reg_wen),
397         .mem_ren_out(mem_wb_mem_ren),
398         .jump_en_out(mem_wb_jump_en),
399         .mem_width_out(mem_wb_mem_width),
400         .load_unsigned_out(mem_wb_load_unsigned)
401     );
402
403     // ============================================================================
404     // WB Stage
405     // ============================================================================
406
407     wire [31:0] mem_read_data_processed;
408
409     // Sign-extension logic matching original
410     assign mem_read_data_processed =
411         (mem_wb_mem_width == `MEM_WIDTH_BYTE) ?
412             (mem_wb_load_unsigned ? {{24{1'b0}}, mem_wb_mem_rdata_raw[7:0]} :
413                                     {{24{mem_wb_mem_rdata_raw[7]}},
    mem_wb_mem_rdata_raw[7:0]}) :
414         (mem_wb_mem_width == `MEM_WIDTH_HALF) ?
415             (mem_wb_load_unsigned ? {{16{1'b0}}, mem_wb_mem_rdata_raw[15:0]} :
416                                     {{16{mem_wb_mem_rdata_raw[15]}},
    mem_wb_mem_rdata_raw[15:0]}) :
417         mem_wb_mem_rdata_raw;
```

```
418
419       // Write back selection
420       assign wb_rd_data = (mem_wb_mem_ren) ? mem_read_data_processed :
421                           (mem_wb_jump_en) ? mem_wb_pc_plus_4 :
422                           mem_wb_alu_result;
423
424       // ============================================================================
425       // Hazard & Forwarding Units
426       // ============================================================================
427
428       ForwardingUnit u_forwarding (
429           .id_ex_rs1(id_ex_rs1),
430           .id_ex_rs2(id_ex_rs2),
431           .ex_mem_rd(ex_mem_rd),
432           .ex_mem_reg_wen(ex_mem_reg_wen),
433           .mem_wb_rd(mem_wb_rd),
434           .mem_wb_reg_wen(mem_wb_reg_wen),
435           .forward_a(forward_a),
436           .forward_b(forward_b)
437       );
438
439       HazardDetectionUnit u_hazard (
440           .id_ex_mem_ren(id_ex_mem_ren),
441           .id_ex_rd(id_ex_rd),
442           .id_rs1(id_rs1),
443           .id_rs2(id_rs2),
444           .id_opcode(id_opcode),
445           .branch_taken_ex(branch_taken_ex),
446           .stall_if(stall_if),
447           .flush_if_id(flush_if_id),
448           .flush_id_ex(flush_id_ex)
449       );
450
451   endmodule
```

Core Functions

- Pipeline Stage Integration: Connects the five pipeline stages (IF, ID, EX, MEM, WB) with corresponding pipeline registers
- Control Flow Management: Manages program counter updates, branch resolution, and instruction flow
- Hazard Resolution: Integrates forwarding and hazard detection units to handle data and control dependencies

- Memory Interface: Provides unified interface to external instruction and data memory systems

Key Components

Pipeline Structure:

- IF Stage: Instruction fetch with PC management
- ID Stage: Instruction decoding and register read
- EX Stage: Execution with ALU operations and branch evaluation
- MEM Stage: Data memory access and load/store operations
- WB Stage: Register write-back

Support Modules:

- ForwardingUnit: Resolves data hazards by bypassing results from later stages
- HazardDetectionUnit: Detects load-use hazards and control hazards, generates stall/flush signals
- Control Unit: Generates pipeline control signals based on instruction type
- Pipeline Registers: Four inter-stage registers (IF/ID, ID/EX, EX/MEM, MEM/WB)

External Interfaces:

- Instruction memory interface for instruction fetch
- Data memory interface for load/store operations
- Global control signals (clock, reset, run enable)

To realize this pipeline, the single-cycle design is extended with inter-stage pipeline registers, hazard detection, and forwarding logic. The following pipeline modules are introduced:

- **Pipe_IF_ID** – Between IF and ID stages.
- **Pipe_ID_EX** – Between ID and EX stages.
- **Pipe_EX_MEM** – Between EX and MEM stages.
- **Pipe_MEM_WB** – Between MEM and WB stages.

Each pipeline register captures the outputs of one stage and delivers them to the next stage on the rising clock edge, provided the pipeline is enabled (`pipe_en = run & ~global_stall`). Control signals such as `flush` and `stall` are used to handle hazards and maintain correctness.

In `CPU_TOP.v` , these pipeline modules are instantiated and interconnected to form the complete datapath. The top module also integrates the hazard detection unit

(`HazardDetectionUnit`) and forwarding unit (`ForwardingUnit`) to resolve data and control hazards.

Pipeline Modules

Pipe_IF_ID

- Inputs: `pc_in` , `inst_in` from the IF stage.
- Outputs: `pc_out` , `inst_out` to the ID stage.
- Control: `hold` stalls the pipeline (e.g., for load-use hazard); `flush` clears the register (e.g., on branch misprediction).
- Role: Preserves the instruction and PC for one cycle, allowing the ID stage to operate on stable values.

代码块

```
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            pc_out <= 32'b0;
            inst_out <= 32'h00000013; // NOP
        end else if (en) begin
            if (flush) begin
                pc_out <= 32'b0;
                inst_out <= 32'h00000013;
            end else if (!hold) begin
                pc_out <= pc_in;
                inst_out <= inst_in;
            end
            // Else hold
        end
    end
```

Pipe_ID_EX

- Inputs: Decoded signals from ID: `pc` , `imm` , `rs1_data` , `rs2_data` , `rd` , `rs1` , `rs2` , `funct3` , and all control signals (`alu_op` , `mem_wen` , `reg_wen` , etc.).
- Outputs: Corresponding signals to the EX stage.
- Control: `flush` injects a bubble (NOP) when a hazard is detected.
- Role: Passes operands and control to the execution stage.

代码块

```
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
```

```
            pc_out <= 0;
            imm_out <= 0;
            rs1_data_out <= 0;
            rs2_data_out <= 0;
            rd_out <= 0;
            rs1_out <= 0;
            rs2_out <= 0;
            funct3_out <= 0;
            alu_op_out <= `ALU_OP_NOP;
            src1_sel_out <= 0;
            src2_sel_out <= 0;
            mem_wen_out <= 0;
            mem_ren_out <= 0;
            reg_wen_out <= 0;
            branch_en_out <= 0;
            jump_en_out <= 0;
            mem_width_out <= 0;
            load_unsigned_out <= 0;
        end else if (en) begin
            if (flush) begin
                // Flush => Insert Bubble
                // Clear control signals that cause side effects
                mem_wen_out <= 0;
                mem_ren_out <= 0;
                reg_wen_out <= 0;
                branch_en_out <= 0;
                jump_en_out <= 0;
                alu_op_out <= `ALU_OP_NOP;
                // Others can hold value or be zero, doesn't matter much
                pc_out <= 0;
                imm_out <= 0;
                rs1_data_out <= 0;
                rs2_data_out <= 0;
                rd_out <= 0;
                rs1_out <= 0;
                rs2_out <= 0;
                funct3_out <= 0;
                src1_sel_out <= 0;
                src2_sel_out <= 0;
                mem_width_out <= 0;
                load_unsigned_out <= 0;
            end else begin
                pc_out <= pc_in;
                imm_out <= imm_in;
                rs1_data_out <= rs1_data_in;
                rs2_data_out <= rs2_data_in;
                rd_out <= rd_in;
```

```
                rs1_out <= rs1_in;
                rs2_out <= rs2_in;
                funct3_out <= funct3_in;

                alu_op_out <= alu_op_in;
                src1_sel_out <= src1_sel_in;
                src2_sel_out <= src2_sel_in;
                mem_wen_out <= mem_wen_in;
                mem_ren_out <= mem_ren_in;
                reg_wen_out <= reg_wen_in;
                branch_en_out <= branch_en_in;
                jump_en_out <= jump_en_in;
                mem_width_out <= mem_width_in;
                load_unsigned_out <= load_unsigned_in;
            end
        end
    end
```

Pipe_EX_MEM

- Inputs: Results from EX: `alu_result`, `rs2_data`, `pc_plus_4`, `rd`, and memory-related control signals.
- Outputs: Signals to the MEM stage.
- Role: Holds results for memory access or later forwarding.

代码块

```

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            alu_result_out <= 0;
            rs2_data_out <= 0;
            pc_plus_4_out <= 0;
            rd_out <= 0;
            mem_wen_out <= 0;
            mem_ren_out <= 0;
            reg_wen_out <= 0;
            jump_en_out <= 0;
            mem_width_out <= 0;
            load_unsigned_out <= 0;
        end else if (en) begin
            alu_result_out <= alu_result_in;
            rs2_data_out <= rs2_data_in;
            pc_plus_4_out <= pc_plus_4_in;
            rd_out <= rd_in;

```

```
            mem_wen_out <= mem_wen_in;
            mem_ren_out <= mem_ren_in;
            reg_wen_out <= reg_wen_in;
            jump_en_out <= jump_en_in;
            mem_width_out <= mem_width_in;
            load_unsigned_out <= load_unsigned_in;
        end
    end
```

Pipe_MEM_WB

- Inputs: Results from MEM: `alu_result`, `mem_rdata`, `pc_plus_4`, `rd`, and write-back controls.
- Outputs: Signals to the WB stage.
- Role: Delivers data to be written back to the register file.

These registers are controlled by a global enable signal (pipe_en) that freezes the pipeline during memory wait states or external stalls.

代码块

```
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            alu_result_out <= 0;
            mem_rdata_out <= 0;
            pc_plus_4_out <= 0;
            rd_out <= 0;
            reg_wen_out <= 0;
            mem_ren_out <= 0;
            jump_en_out <= 0;
            mem_width_out <= 0;
            load_unsigned_out <= 0;
        end else if (en) begin
            alu_result_out <= alu_result_in;
            mem_rdata_out <= mem_rdata_in;
            pc_plus_4_out <= pc_plus_4_in;
            rd_out <= rd_in;

            reg_wen_out <= reg_wen_in;
            mem_ren_out <= mem_ren_in;
            jump_en_out <= jump_en_in;
            mem_width_out <= mem_width_in;
            load_unsigned_out <= load_unsigned_in;
        end
    end
```

Forwarding Unit

To avoid stalls due to data dependencies, a forwarding unit is implemented (ForwardingUnit.v). It detects when a result produced in a later stage is needed by an instruction currently in the EX stage and forwards the data directly.

代码块

```
 // ForwardA logic
    always @(*) begin
        // Default: No forwarding (use ID/EX reg value)
        forward_a = 2'b00;

        // Priority 1: Forward from EX/MEM pipeline register (MEM stage)
        // Occurs when performing arithmetic with result of previous instruction
        if (ex_mem_reg_wen && (ex_mem_rd != 5'b0) && (ex_mem_rd == id_ex_rs1)) begin
            forward_a = 2'b01;
        end
        // Priority 2: Forward from MEM/WB pipeline register (WB stage)
        // Occurs when performing arithmetic with result of 2nd previous instruction
        // AND not covered by EX/MEM hazard
        else if (mem_wb_reg_wen && (mem_wb_rd != 5'b0) && (mem_wb_rd == id_ex_rs1)) begin
            forward_a = 2'b10;
        end
    end

    // ForwardB logic
    always @(*) begin
        forward_b = 2'b00;

        if (ex_mem_reg_wen && (ex_mem_rd != 5'b0) && (ex_mem_rd == id_ex_rs2)) begin
            forward_b = 2'b01;
        end
        else if (mem_wb_reg_wen && (mem_wb_rd != 5'b0) && (mem_wb_rd == id_ex_rs2)) begin
            forward_b = 2'b10;
        end
    end
```

- If the destination register of an instruction in the EX/MEM stage (`ex_mem_rd`) matches a source register of the instruction in the ID/EX stage (`id_ex_rs1` or `id_ex_rs2`), and the EX/MEM instruction will write back (`ex_mem_reg_wen`), forward the result from EX/MEM (`forward_a = 01` or `forward_b = 01`).
- Similarly, if the match is with the MEM/WB stage (`mem_wb_rd`), forward from MEM/WB (`forward_a = 10` or `forward_b = 10`).
- Forwarding priority: EX/MEM > MEM/WB.

Implementation in CPU_TOP:

The forwarded data is multiplexed into the ALU inputs:

代码块

```
assign ex_rs1_final = (forward_a == 2'b01) ? mem_forward_data :
                      (forward_a == 2'b10) ? wb_rd_data :
                      id_ex_rs1_data;
```

This eliminates stalls for most RAW (Read-After-Write) hazards.

Hazard Detection Unit

The `HazardDetectionUnit.v` detects two primary hazards:

代码块

```
    // R-type, I-type (including Load, JALR), S-type, B-type use RS1
    // LUI (0110111), AUIPC (0010111), JAL (1101111) do NOT use RS1
    assign use_rs1 = (id_opcode != 7'b0110111) && // LUI
                     (id_opcode != 7'b0010111) && // AUIPC
                     (id_opcode != 7'b1101111);   // JAL

    // R-type (0110011), S-type (0100011), B-type (1100011) use RS2
    assign use_rs2 = (id_opcode == 7'b0110011) || // R-type
                     (id_opcode == 7'b0100011) || // Store
                     (id_opcode == 7'b1100011);   // Branch

    wire load_use_hazard;
    // Load-Use Hazard Condition:
    // 1. Instruction in EX is a Load (id_ex_mem_ren is true)
    // 2. Destination of Load matches Source of ID instruction
    // 3. ID instruction actually uses that source (and source != x0)
    assign load_use_hazard = id_ex_mem_ren &&
                             (
                               (use_rs1 && (id_rs1 != 5'b0) && (id_ex_rd ==
id_rs1)) ||
```

```
                                (use_rs2 && (id_rs2 != 5'b0) && (id_ex_rd == id_rs2))
                            );

    always @(*) begin
        // Default values
        stall_if = 1'b0;
        flush_if_id = 1'b0;
        flush_id_ex = 1'b0;

        // Load-Use Hazard: Stall IF and ID, Flush EX (inject bubble)
        if (load_use_hazard) begin
            stall_if = 1'b1;      // Stall PC and IF/ID
            flush_id_ex = 1'b1;   // Flush ID/EX
        end

        // Control Hazard: Branch Taken in EX
        // Flush IF/ID (instruction fetched) and ID/EX (instruction in decode)
        // Original reference flushed IF and ID stages.
        if (branch_taken_ex) begin
            flush_if_id = 1'b1;
            flush_id_ex = 1'b1;
        end
    end
```

Load-Use Hazard

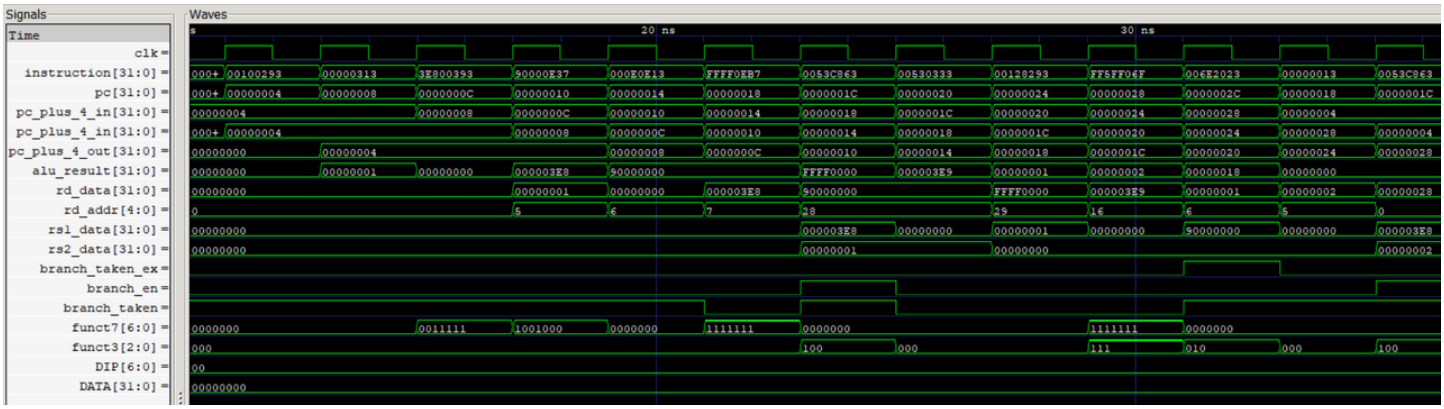
- Occurs when a load instruction (`id_ex_mem_ren`) is followed by an instruction that uses the loaded data.
- Detection: If `id_ex_rd` matches `id_rs1` or `id_rs2` of the current ID instruction, and the ID instruction actually uses that register.
- Action: Stall the IF and ID stages (`stall_if = 1`), flush the ID/EX pipeline register (`flush_id_ex = 1`) to insert a bubble.

Control Hazard (Branch/Jump)

- Occurs when a branch or jump is taken (`branch_taken_ex`).
- Action: Flush the IF/ID and ID/EX registers (`flush_if_id = 1` , `flush_id_ex = 1`) to discard incorrectly fetched instructions.

The unit also considers instruction types to avoid false stalls (e.g., LUI, AUIPC, JAL do not use RS1).

RISC-V five-stage pipeline CPU simulation waveform diagram



From the waveform, it is clearly visible that the PC signal is passed step by step through the five-stage pipeline, the instruction is correctly decoded into funct7 and funct3 signals, the alu_result signal corresponds to the correct value, and data is correctly written back to and read from the registers. In the same cycle, each stage of the pipeline is in its correct operating state. It can also be seen that after executing a branch instruction, the PC correctly jumps back.

The comparison of the on-board test results will be presented and discussed in the next chapter.

2.4 Optimization of multi-cycle CPU of multi-cycle RISC-V (RV32I) CPU

In order to further optimize CPU performance and improve CPU running speed, we attempted to add branch prediction functionality to a five-stage pipeline CPU to reduce the time loss during branch instruction jumps.

First, we tried static branch prediction using the BTFN (Backwards Taken, Forwards Not-Taken) strategy. The code implementation is as follows.

代码块

```
// Static Branch Prediction Logic (BTFN - Backward Taken, Forward Not Taken)
// Only predict conditional branches (B-Type).
// Note: Jump instructions (J-Type/I-Type) are not predicted here, resolved in EX (detected as mispredict).

wire id_is_branch = (id_opcode == 7'b1100011);      // 检测B型指令
wire id_imm_sign  = id_imm[31];                     // 立即数符号位
assign id_predict_taken = id_is_branch && id_imm_sign;  // BTFN策略：立即数为负(向后跳)则预测taken
assign id_branch_target = if_id_pc + id_imm;        // 计算预测的分支目标地址


// Select next PC:
// 1. EX Misprediction Recovery (Highest Priority)
// 2. ID Static Prediction (Taken)
// 3. PC+4 (Default)

```

```
assign pc_next = (ex_mispredict && branch_taken_ex)  ? branch_target :
                 // 优先级1: EX检测到错预测，且实际跳转 → 跳到正确目标

                 (ex_mispredict && !branch_taken_ex) ? (id_ex_pc + 4) :
                 // 优先级2: EX检测到错预测，实际不跳转 → 下一条顺序指令

                 (id_predict_taken)                  ? id_branch_target :
                 // 优先级3: ID预测跳转 → 使用预测的目标地址

                 pc_plus_4;                          // 优先级4: 默认 PC

wire branch_condition_met;
BranchUnit u_branch_unit (
    .rs1_data(ex_rs1_final),
    .rs2_data(ex_rs2_final),
    .funct3(id_ex_funct3),
    .branch_taken(branch_condition_met)      // 实际的分支结果
);

// Detect Misprediction
// 1. Conditional Branch: Actual Taken != Predicted
// 2. Jump: Always Taken (treated as Mispredict because we don't predict Jumps in ID)
assign ex_mispredict = (id_ex_branch_en && (branch_condition_met != id_ex_predict_taken)) || id_ex_jump_en;
//                                                  ↑ 实际结果       ↑ 预测结果
//                      如果两者不同，则发生错预测

// Branch/Jump taken signal (Resolves Control Hazard)
assign branch_taken_ex = (id_ex_branch_en && branch_condition_met) || id_ex_jump_en;

// Target Calculation
assign branch_target = (id_ex_jump_en) ? alu_result : (id_ex_pc + id_ex_imm);

// 错预测信号触发PC恢复（见IF阶段的pc_next逻辑）

// 在EX阶段与实际结果比较
assign ex_mispredict = (id_ex_branch_en && (branch_condition_met != id_ex_predict_taken))
                       || id_ex_jump_en;
```

Then we tried using dynamic branch prediction by adding a Branch Prediction Unit and using a two-bit predictor. The code implementation is as follows.

```
    // Number of entries in PHT/BTB
    // Reduced from 8 to 5 to save Flip-Flop resources (32 entries)
    // 32 * 34 bits = 1088 FFs, leaving room for other logic.
    localparam INDEX_WIDTH = 5;
    localparam NUM_ENTRIES = 1 << INDEX_WIDTH;

    // Pattern History Table (2-bit saturating counters)
    // 00: Strongly Not Taken
    // 01: Weakly Not Taken
    // 10: Weakly Taken
    // 11: Strongly Taken
    reg [1:0] pht [NUM_ENTRIES-1:0];

    // Branch Target Buffer
    reg [31:0] btb [NUM_ENTRIES-1:0];

    // Index calculation (ignoring byte offset)
    wire [INDEX_WIDTH-1:0] if_index = if_pc[INDEX_WIDTH+1:2];
    wire [INDEX_WIDTH-1:0] update_index = update_pc[INDEX_WIDTH+1:2];

    // Read Prediction
    wire [1:0] counter = pht[if_index];
    assign pred_taken = counter[1]; // MSB 1 implies Taken (10 or 11)
    assign pred_target = btb[if_index];

    integer i;
    // Update Logic
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            for (i = 0; i < NUM_ENTRIES; i = i + 1) begin
                pht[i] <= 2'b00; // Reset to Strongly Not Taken
                btb[i] <= 32'b0;
            end
        end else if (update_en) begin
            // Update BTB only if actually taken (to learn the target)
            // Or always update? If not taken, target is irrelevant, but
keeping it old is fine.
            if (update_taken) begin
                btb[update_index] <= update_target;
            end

            // Update Saturating Counter
            case (pht[update_index])
                2'b00: pht[update_index] <= update_taken ? 2'b01 : 2'b00; //
SNT -> WNT / SNT
                2'b01: pht[update_index] <= update_taken ? 2'b10 : 2'b00; //
WNT -> WT / SNT
```

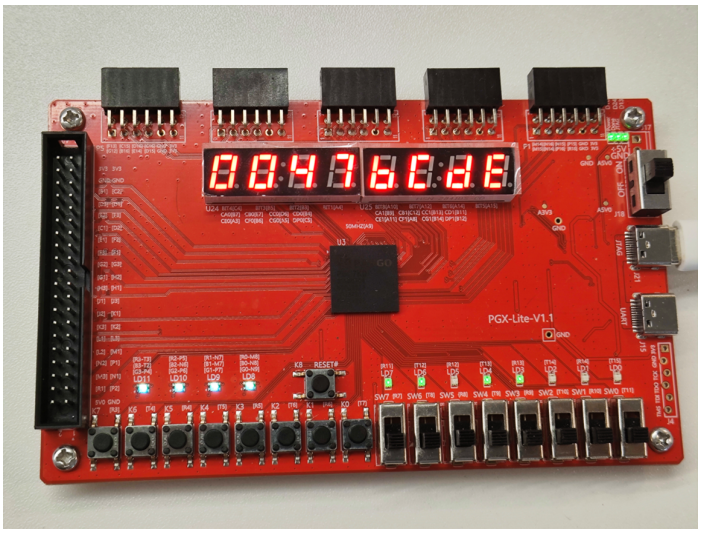
```
                2'b10: pht[update_index] <= update_taken ? 2'b11 : 2'b01; // WT -> ST / WNT
                2'b11: pht[update_index] <= update_taken ? 2'b11 : 2'b10; // ST -> ST / WT
            endcase
        end
    end
```

代码块

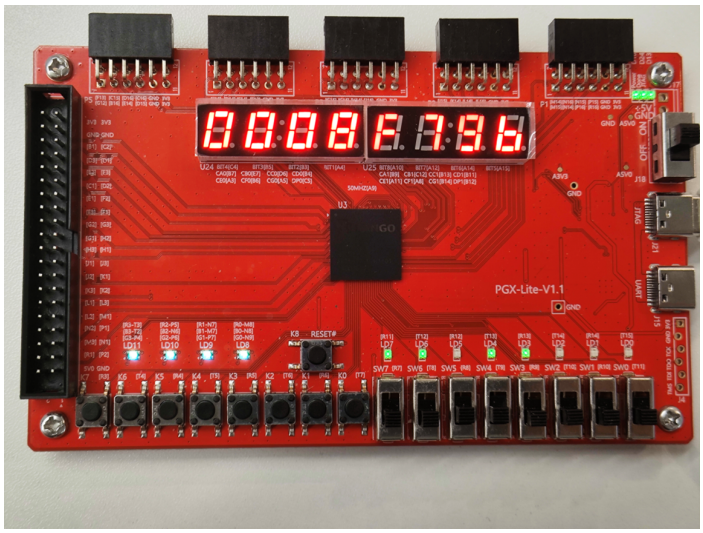
```
    // Select next PC logic:
    // 1. If misprediction in EX stage (flush), go to corrected PC.
    // 2. Else if taken prediction in IF, go to predicted target.
    // 3. Else PC+4.
    assign pc_next = flush_ex_mispredict ? correct_pc_ex :
                     (pred_taken_if ? pred_target_if : pc_plus_4);
```

During the coding process of the dynamic branch predictor, we encountered insufficient Flip-Flop resources during compilation. The PDS development board we used has only 8,800 Flip-Flop resources. However, in the initial version of the BranchPredictionUnit, we employed a prediction table with an 8-bit index width, which led to the shortage of FF resources. After multiple attempts, we found that the most suitable prediction table index length is 5 bits. Using a larger index length would cause compilation clock non-convergence and resource insufficiency, while a smaller index length would reduce prediction effectiveness.

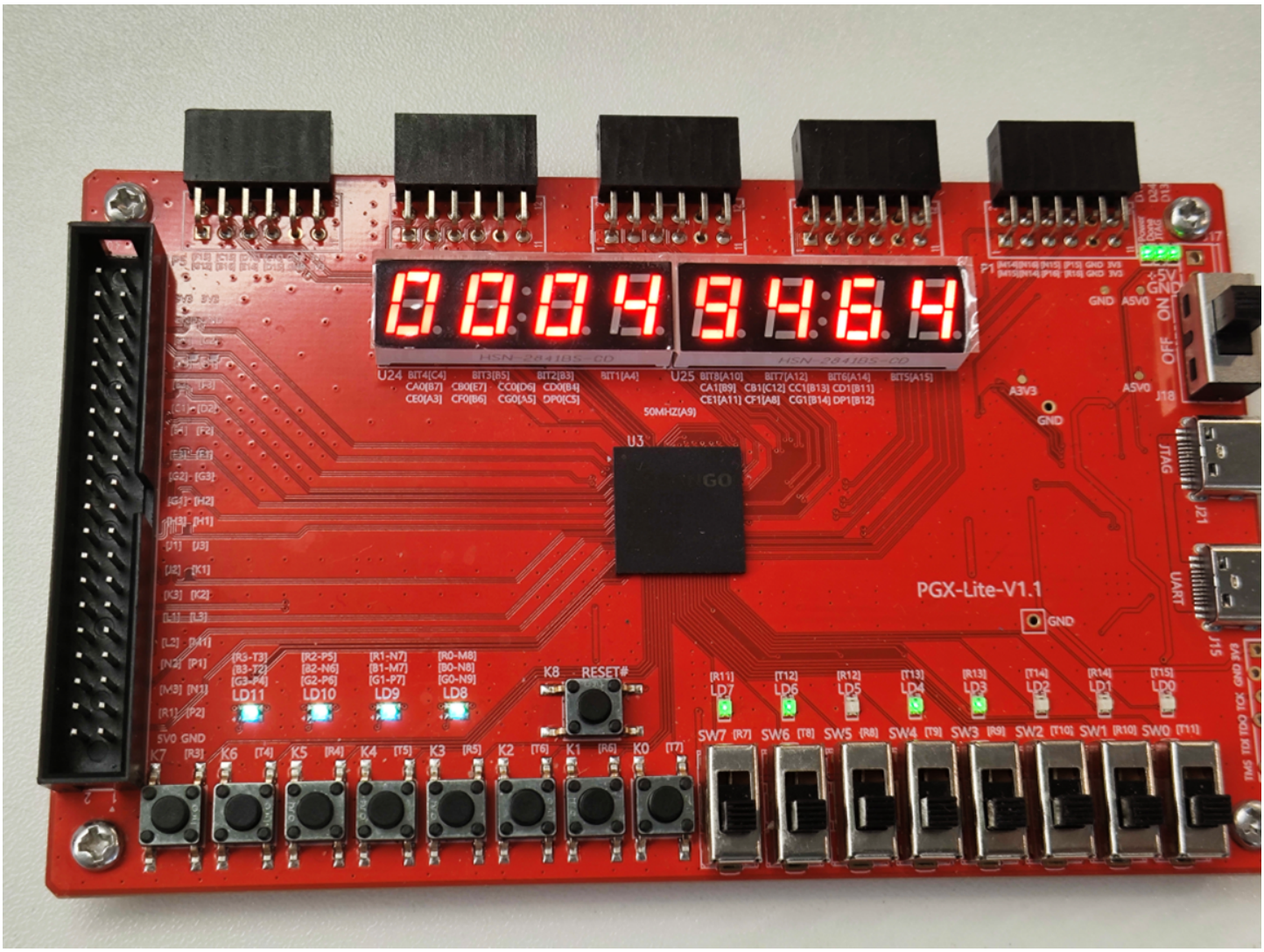
Comparison of Timing Effects on the Board



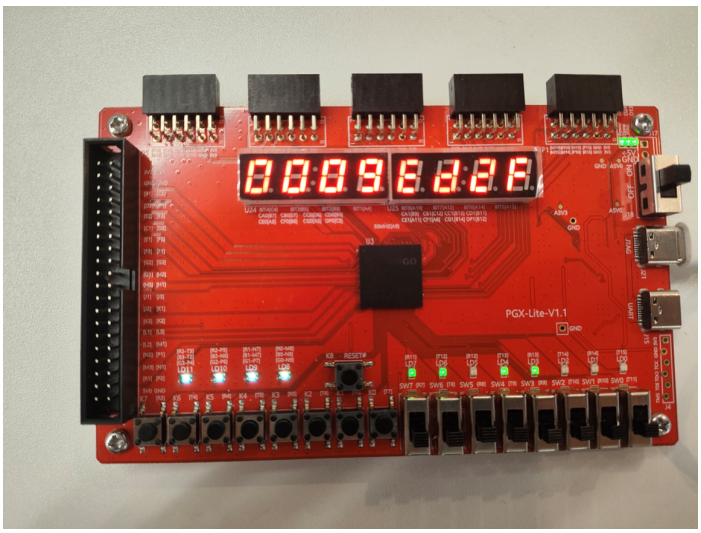
single_cylcle_CPU(clock_freq-5MHz)



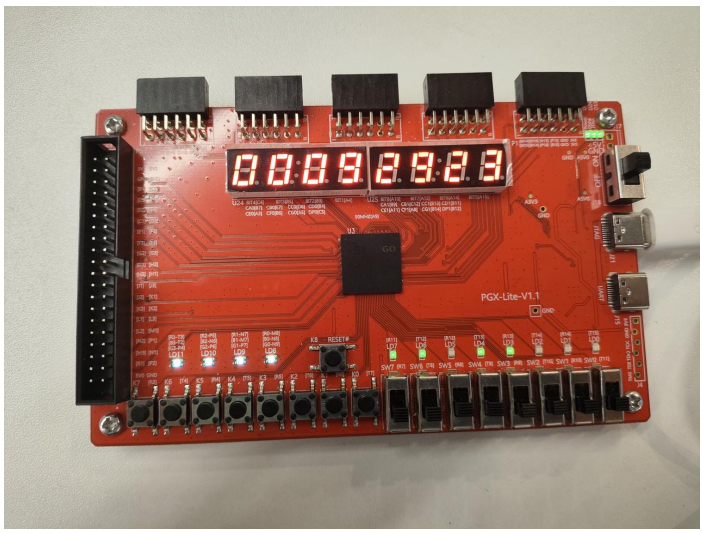
single_cylcle_CPU(clock_freq-40MHz)



pipeline_CPU with static branch prediction(clock_freq-100MHz)

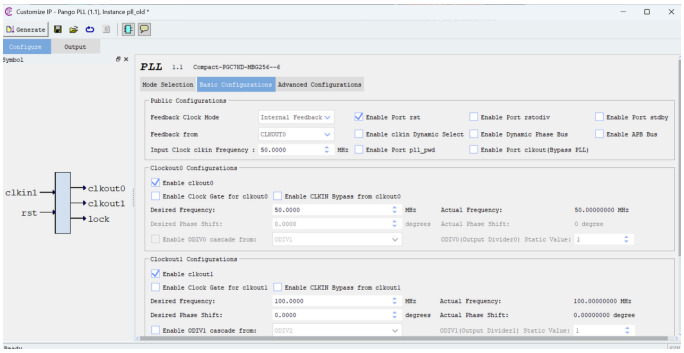


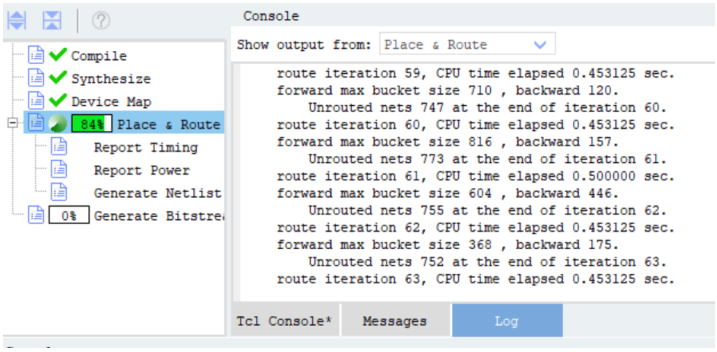
pipeline_CPU with dynamic branch prediction(clock_freq-40MHz)



pipeline_CPU with dynamic branch prediction(clock_freq-40MHz)-reset

During hardware testing, we discovered that if the clock frequency set during compilation is too high, even if there are no logical errors in the code, the compiler will get stuck at 84% during the Place & Route step. The reason is that the excessive clock frequency causes the clock period to be shorter than the step with the longest delay within the cell.





Single-cycle CPU: At a clock frequency of 5MHz, the execution time is **0x47BCDE**. After multiple attempts, the maximum clock frequency for the single-cycle CPU was found to be approximately 40MHz, with an execution time of **0x8F79B**, which is exactly one-eighth of the time at 5MHz, matching the prediction.

When we introduced pipelining and BTFN static branch prediction: After multiple attempts, the maximum clock frequency for the pipeline_CPU with static branch prediction was found to be approximately **100MHz**, with an execution time of **0x49464**, showing significant improvement.

When we introduced pipelining and BTFN dynamic branch prediction: After multiple attempts, the maximum clock frequency for the pipeline_CPU with dynamic branch prediction was found to be only about 40MHz. The reason for this being only 40MHz is speculated to be due to the relatively complex logic of dynamic branch prediction, where the state machine of the two-bit branch predictor, operating with read/write operations in the prediction table, introduces longer latency. The execution time was **0x9ED2F**, slightly longer than that of the single-cycle CPU at the same clock frequency, possibly due to the longer latency introduced by branch prediction. Additionally, we found that after running the program once on the board and pressing the reset button to run it again, the execution time was somewhat reduced. This is analyzed to be because after the first run, the prediction table and the two-bit predictor were already set up, saving the prediction adjustment behavior, which further verifies the success of our dynamic branch prediction design.

2.5 Implementation challenges and reflections

The above-mentioned difficulties have been well reflected in the report. Additionally, when adjusting the waveform diagram, it was observed that there was a problem with the PC after the blt instruction jump. By tracing back step by step according to the waveform diagram, it was found that the values of the two sources calculated by the ALU were incorrect. After the modification, the waveform diagram could output correctly. During the debugging on the board, the add.loop file was adjusted step by step based on the given instruction set errors, and the values of irom and dram were appropriately modified. For instance, there were difficulties such as the comparison between signed and unsigned numbers, and the load-type instructions needed to pause for one cycle to ensure correct reading, etc.

When designing a pipeline, the logic of each segment needs to maintain a relatively high quality level in order to achieve significant performance improvement. If the logic of a certain part is overly complex, it will consume too many clock cycles to complete, resulting in a significant decline in the overall pipeline performance. The simpler the language, the more ingenious the thinking it contains can benefit people. For example, the comparison of two numbers without using "sign" avoids hardware incompatibility and also reduces variable design.

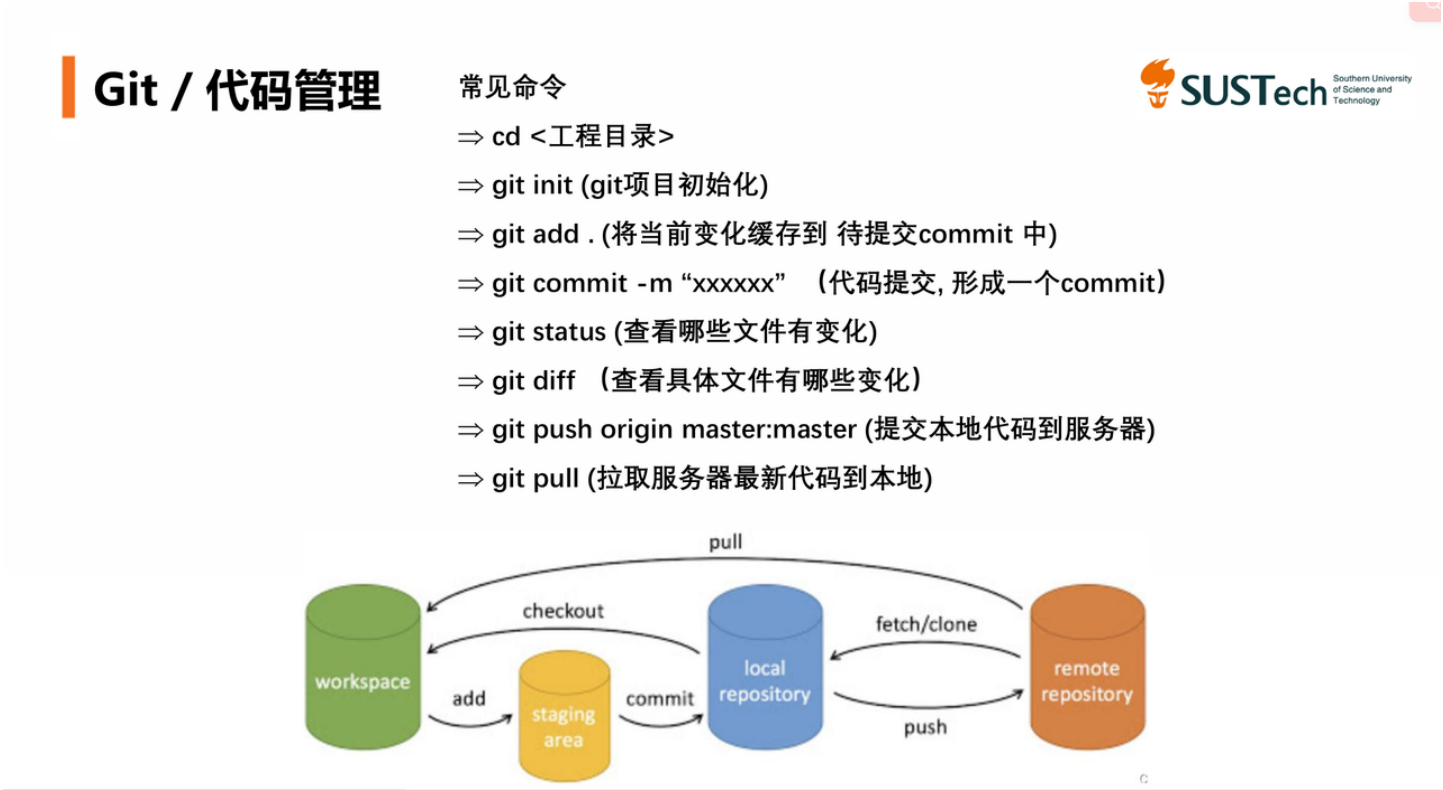
In the design phase of the multi-cycle CPU state machine, we also encountered a problem where the execution processes of different instructions in RV32I varied significantly (for example, R-type instructions only require 4 cycles, while lw requires 5 cycles). The state jump logic was prone to deadlock or incomplete instruction execution. To solve this, I first analyzed the execution steps for each instruction type (R/I/S/B/U/J), drew waveform diagrams, and then conducted module-based analysis to trace the paths of each signal source and gradually debug. This process taught me that for the design of complex control logic, the modular approach of 'first splitting the instruction scenarios and then merging the common logic' is beneficial.

3. Bonus

For our project's version control and collaborative development, we adopted Gitee, a leading Git-based code hosting platform in China. Often described as the Chinese counterpart to GitHub, Gitee provides a comprehensive suite of tools for software development, including repository management, issue tracking, pull requests, wikis, and continuous integration/continuous deployment (CI/CD) pipelines via its Gitee Go service. Its primary advantage lies in its localization; it offers significantly faster access speeds within China and a fully Chinese-language interface, which reduces barriers for the entire team. Furthermore, it complies with local data regulations, ensuring project security.

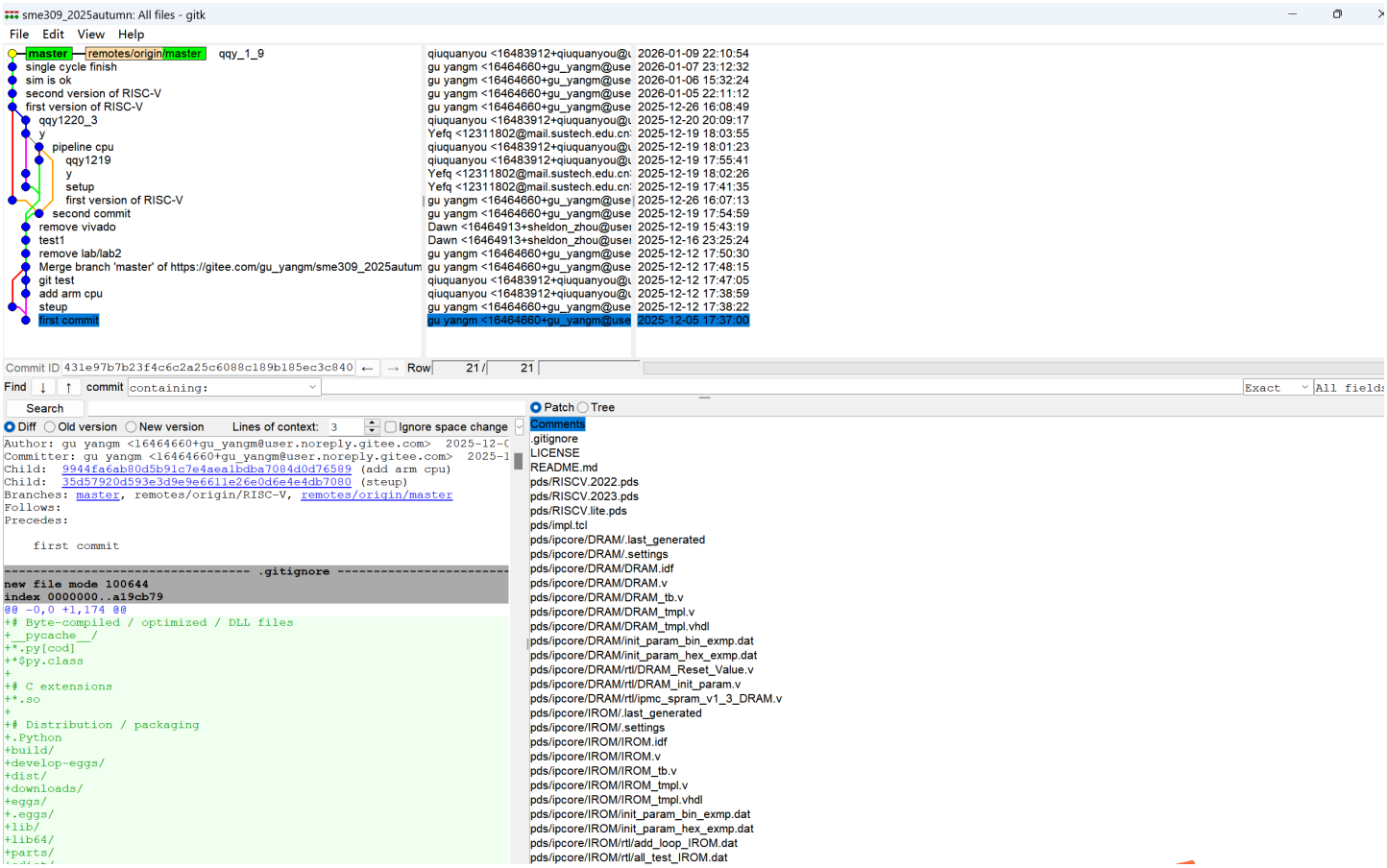
The use of Gitee brought multiple tangible benefits to our group collaboration. Firstly, it centralized our codebase, serving as a single source of truth and eliminating the confusion of managing disparate code versions. Secondly, its branch and pull request workflow enabled structured and conflict-free parallel development. Team members could work on independent features in separate branches and then merge them into the main branch through reviewed pull requests, greatly enhancing code quality and team coordination. Thirdly, features like issue tracking and milestone management allowed us to efficiently break down project tasks, assign responsibilities, and monitor overall progress in a transparent manner. Finally, the platform's built-in wiki facilitated the creation and maintenance of project documentation, ensuring knowledge was effectively shared and preserved. In summary, Gitee was instrumental in streamlining our development workflow, improving communication, and fostering a disciplined, professional approach to collaborative coding.

The code download and upload on Gitee are mainly done through the terminal, and the main commands used are as follows:

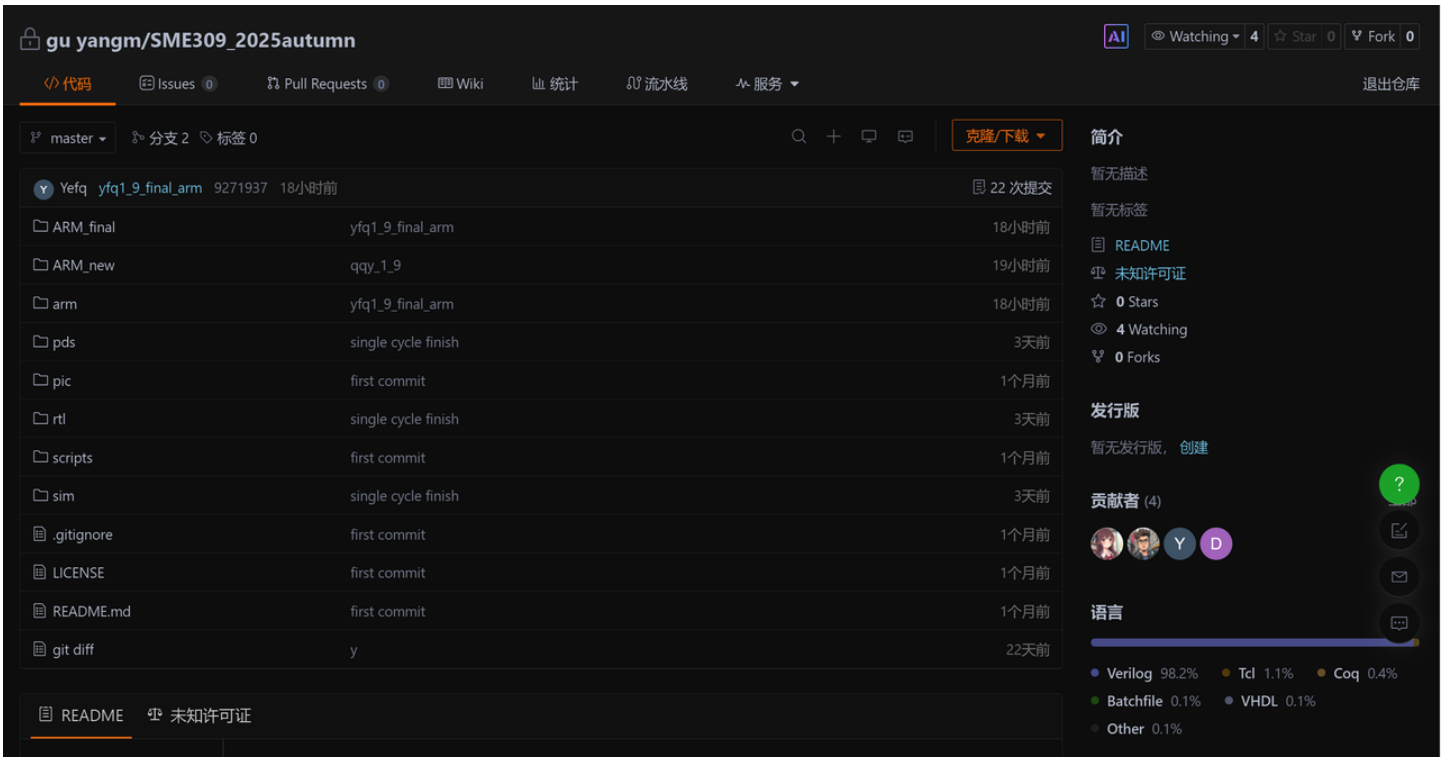


Among the key aspects of the usage experience is the "git pull" command. This command enables the update of the code newly uploaded by teammates. If the latest code of the teammates is not updated before uploading, it will result in the local system not being synchronized with the cloud, and ultimately the upload will fail.

Another useful command is gitk. This can open the working repository graph, making the work structure very clear and further enhancing the collaboration efficiency.



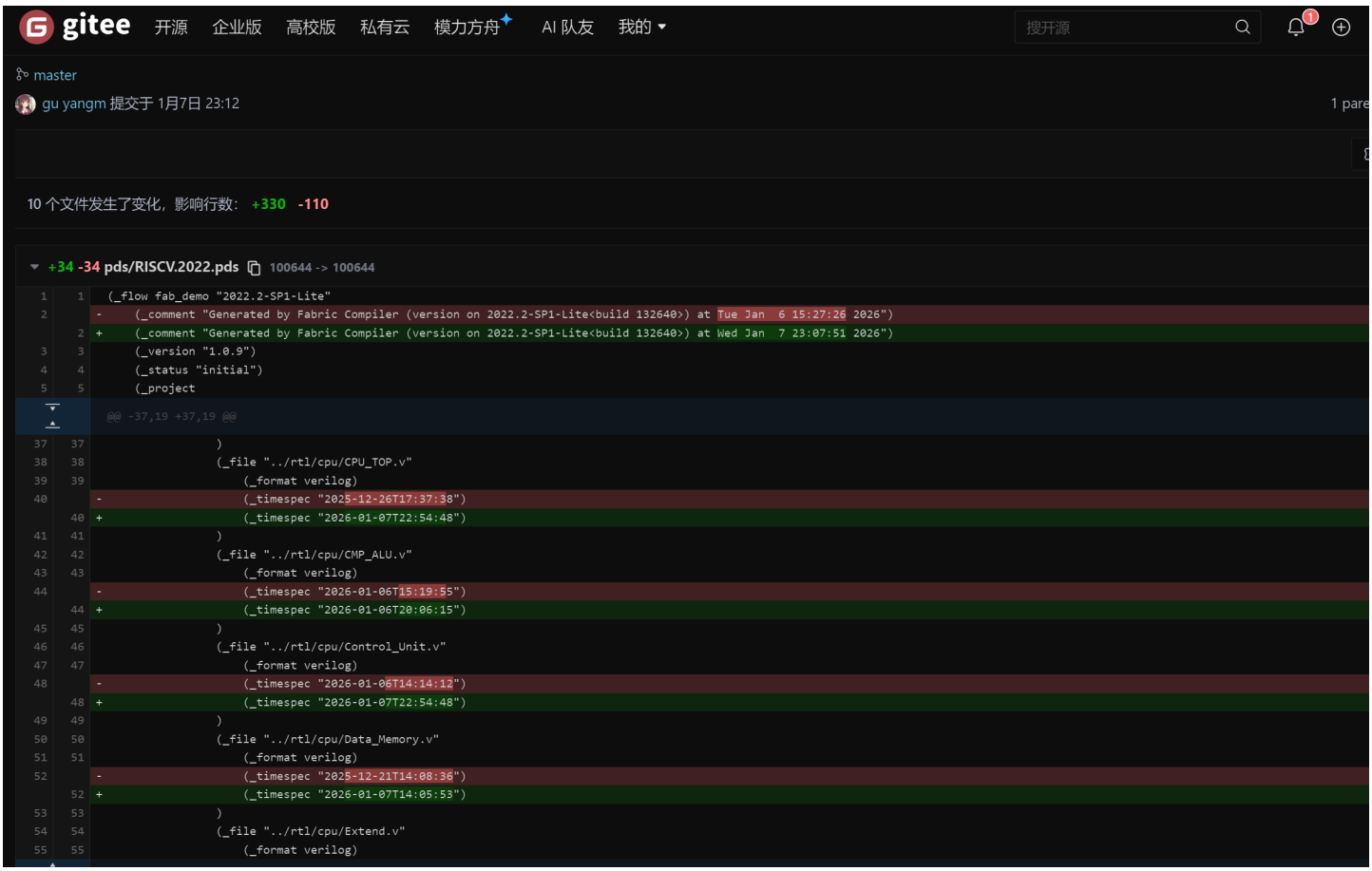
Next, we will show the files in our library.



The first three files are related to ARM, like arm, ARM_new, ARM_final. They were shared among each other on Gitee, enabling the two students to work collaboratively and successfully complete the ARM part. During the process, due to the change of the position of condlogic, it was moved from the ID stage to the EX stage to update the condition codes. This operation

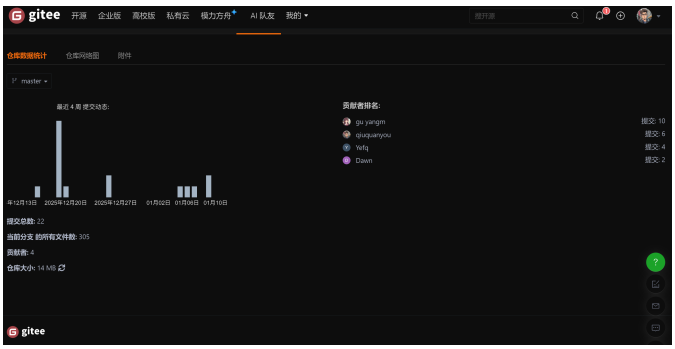
significantly changed the structure of the file, so multiple folders were created. However, thanks to the existence of gitee, the modification was still relatively easy.

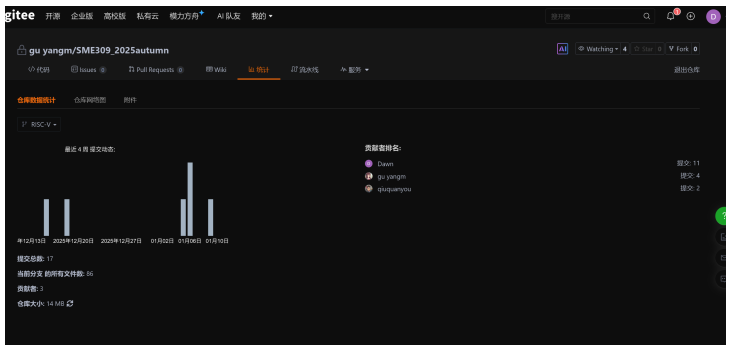
The remaining files are mainly related to RISC_V. The modification of the sim file effectively demonstrates the significant advantage of Gitee, which enables users to see exactly where the code has been modified.

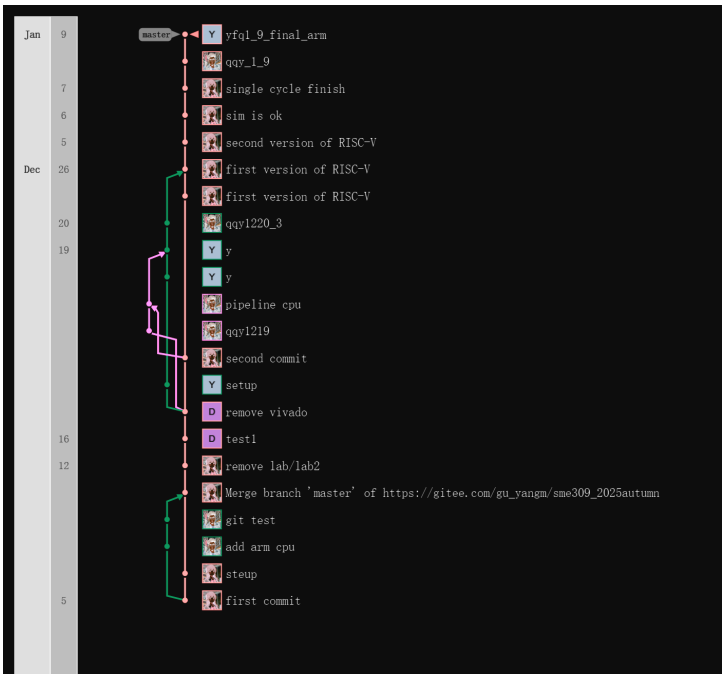


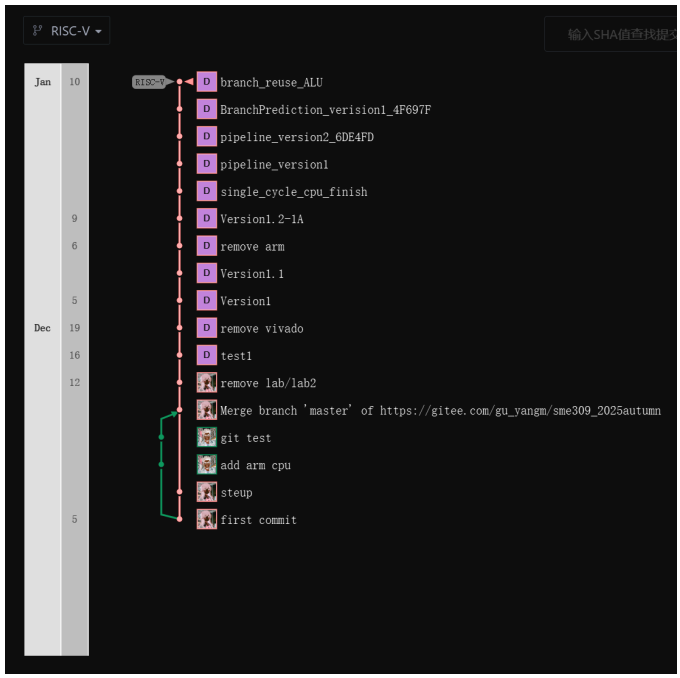
The red code lines represent the removed code, while the green ones represent the newly added code lines. When the team members discover that the newly uploaded code has bugs, they can easily identify the problem areas in the code and notify the teammate who uploaded the code to make the necessary modifications, thereby significantly improving the collaboration efficiency.

Finally, the contribution graph is presented：









The above picture roughly shows the upload situation of our group. However, the number of uploads is relatively small, which does not mean that the teammates' contributions are low. The teammates with fewer uploads prefer to upload code that has been revised many times, which is the reason for the low number of uploads. Overall, the team leader who built the database made the greatest contribution, resolving the most challenging aspects of RISC_V. The other three team members made average contributions.

In summary, Gitee has indeed effectively facilitated the transfer of code among the four team members, effectively coordinated the cooperation issues of the project, and improved efficiency. I am very grateful to Gitee.